

Solving Mixed-Integer Problems as QUBO: Encodings, Reformulations, and Rolling-precision Algorithm

Ashfaq Iftakher¹, Metin Türkay² and M. M. Faruque Hasan^{1,3,*}

¹Artie McFerrin Department of Chemical Engineering, Texas A&M University, College Station,
TX 77843-3122, USA.

²Koç University, Department of Industrial Engineering, Sarıyer, Istanbul 34450, Turkey.

³Texas A&M Energy Institute, Texas A&M University, College Station, TX, 77843, USA.

Abstract

Many real-world optimization problems are constrained mixed-integer programs. This is problematic for quantum annealers and quadratic unconstrained binary optimization (QUBO)-based quantum optimization platforms, that operate natively on binary quadratic models. We present a unified framework for solving mixed-integer problems through QUBO. We present new encoding schemes that represent bounded continuous variables and selected nonlinear terms using structured collections of binary variables with controllable decimal precision. We review how a broad class of mixed-integer and geometric programs can be reformulated as QUBO. We further introduce a rolling-precision algorithm that avoids a monolithic high-resolution QUBO by solving a sequence of smaller QUBOs on successively refined discretization grids. Two strategies are proposed: (i) sequential bit-growth that progressively increases discretization depth, and (ii) constant-size zoom-in scheme that fixes the number of bits and refines precision by tightening variable bounds around incumbents. Illustrative tests demonstrate solutions with controllable accuracy while keeping individual QUBO sizes moderate.

Keywords: QUBO, Mixed-integer program, Encoding, rolling-precision

*Correspondence should be addressed to M.M. Faruque Hasan at hasan@tamu.edu, Tel.: +1 979-862-1449.

1 Introduction

Quantum computing (QC) offers a fundamentally different paradigm for computation by exploiting superposition, entanglement, and tunneling phenomena in quantum systems (Albasha & Lidar 2018, Nielsen & Chuang 2002). Unlike classical bits that take values in $\{0, 1\}$, quantum bits (qubits) can exist in coherent superpositions of states, potentially enabling asymptotic speedups for certain classes of problems (Coppersmith (2002), Deutsch & Jozsa (1992)). Well-known examples include Shor’s factoring algorithm and Grover’s unstructured search (Grover 1996, Shor 1999). In the context of optimization, an increasing body of work focuses on *quantum annealing* (QA) and related adiabatic quantum computation schemes, which aim to find low-energy states of Ising Hamiltonians (Albasha & Lidar 2018, Grant & Humble 2020, Hauke et al. 2020, Tanaka et al. 2017, Yarkoni et al. 2022). While finding the ground state is NP-hard (Barahona (1982)), quantum properties are expected to enable quantum optimization algorithms to rapidly explore a vast solution space (Baldassi & Zecchina (2018), Lanting et al. (2014), Razavy (2013)), thereby providing good solutions within computationally tractable time. Through standard mappings, many discrete optimization problems can be expressed as a quadratic unconstrained binary optimization (QUBO) problem:

$$\min_{\mathbf{z} \in \{0,1\}^n} \mathbf{z}^\top \mathbf{Q} \mathbf{z}, \tag{1}$$

where $\mathbf{Q}$ is a symmetric matrix, and $\mathbf{z}$ is a vector of binary variables. This QUBO model is amenable to QA-based solution and is ubiquitous in many NP-hard combinatorial optimization problems (Boros & Hammer 2002, Lucas 2014, 2019). For example, numerous problems such as max-cut, quadratic assignment, set partitioning, graph coloring, and various scheduling formulations admit natural QUBO encodings (Glover et al. 2019, Kochenberger et al. 2014).

Despite this modeling flexibility, several obstacles arise when attempting to use QUBO and QA to solve optimization problems that are common in engineering domains. First, many real-world problems, particularly in process systems engineering, energy systems, and supply chain planning, are formulated as mixed-integer linear or quadratic programs (MILPs, MIQPs, MIQCQPs) with both continuous and integer variables and with linear and nonlinear constraints (Bernal et al. 2022, Biegler et al. 1997, Iftakher et al. 2025,

Iftakher, Monjur & Hasan 2023). However, quantum annealers and most QUBO-based solvers operate on binary variables. As a result, continuous domains or constraints cannot be enforced natively (Ishikawa (2010), Valiante et al. (2021)). Second, current quantum hardware platforms are limited in qubit count, connectivity, and control precision (Hauke et al. 2020, Murali et al. 2019). Embedding a large dense QUBO into hardware connectivity graphs such as Chimera or Pegasus typically incurs overhead that grows superlinearly with the number of logical variables (Boothby et al. 2020, Cai et al. 2014). As a result, directly encoding large-scale mixed-integer programs as a single QUBO is not yet feasible in practice.

Regarding encoding schemes, existing QUBO encodings typically represent a bounded discrete variable ($k \in \{0, \dots, K\}$) directly as a single block of binary variables. The binary encoding is the most compact in bit count, whereas unary and one-hot encodings consider a tradeoff of compactness for simpler coefficient structure (Tamura et al. (2021)). Domain-wall encoding (Chancellor (2019)) reduces one-hot’s intra-variable connectivity while preserving two-body interaction realizations. Bounded-coefficient encoding (Karimi & Ronagh (2019)) handles encoding coefficients to improve robustness to hardware noise. Arithmetic-progression encoding (Xavier et al. (2023)) provides an intermediate ($O(\sqrt{K})$) compromise between binary compactness and coefficient moderation (See Xavier et al. (2023) for details). Regarding iterative refinement of solutions, several works recommended the adoption of the iterative refinement algorithm to solve classical numerical analysis problems. In order to solve the systems of linear equations, a mixed-precision algorithm was proposed (Koska et al. 2025). The proposed algorithm solves the problem in a hybrid classical-quantum scheme that first solves a problem in low precision (i.e., 10^{-8}), and takes this solution to refine it afterwards to a double precision solution (i.e., 10^{-16}). This approach is limited to very small-scale problems since encoding each digit of a single-precision problem requires very large quantum architectures. (Mohammadisiahroudi et al. 2025) proposed a quantum computing-inspired iterative refinement algorithm for semidefinite optimization problems where both feasibility and optimality gaps are progressively addressed in an iterative setting. The values of decision variables are updated to reduce the error tolerance to the linear optimization problem by using the primal-dual relationship of linear programming problems without controlling the number of digits of the decision variables. In both of these early adoptions of iterative refinement for quantum computing, the control of the error tolerance or precision is the main

consideration. Hybrid classical–quantum approaches attempt to mitigate these issues by combining classical decomposition or cutting-plane methods with quantum optimization subroutines. For example, Benders-type decompositions and branch-and-bound variants have been coupled with quantum annealing to handle certain classes of MIPs (Ajagekar et al. 2022, 2020, Booth et al. 2017, Chang et al. 2020, Zhao et al. 2022). In such schemes, the quantum device typically solves subproblems involving only binary variables, while the classical solver manages continuous relaxations, cutting planes, or branching decisions. Although promising, these hybrid methods do not fully exploit the potential of QUBO-based optimization as a standalone framework, and they inherit much of the combinatorial complexity of classical algorithms.

In this work, we study how a collection of QUBO modeling ingredients can be assembled into a single mixed-integer-to-QUBO workflow and be paired with a rolling-precision scheme that gradually refines the solution precision. Specifically, we study the following questions: (i) How can bounded continuous and integer variables, as well as selected nonlinear expressions, be represented by binary variables with controllable decimal precision using compact encoding schemes? (ii) How can mixed-integer quadratic programs (MIQPs), mixed-integer quadratically constrained quadratic programs (MIQCQPs), and geometric programs in monomial form be systematically reformulated as a single QUBO problem using such encodings and appropriate penalty functions? and (iii) Given that encodings may lead to extremely large QUBO instances, how can we design an algorithm that keeps individual QUBO sizes manageable while still converging to a high-precision solution? Building on our previous works (Iftakher, Kazi & Hasana (2023), Iftakhera & Hasana (2024)), the main contributions of this work are as follows:

- We present new encoding schemes that map bounded continuous variables and specific nonlinear terms to collections of binary variables with a controllable decimal precision. These encodings include a compact digit-wise specific binary encoding (SBE) and extend naturally to certain nonlinear functions via precomputed coefficients. Rather than proposing a direct encoding of a bounded integer variable, we study bounded continuous and slack variables by first scaling them to $[0, 1]$, discretizing them on a J -digit decimal grid, and then encoding the decimal digits themselves. This yields a digit-sum unary representation using $(9J + 1)$ bits and a compact digit-wise specific binary encoding using $(4J + 1)$ bits per continuous/slack variable. Hence,

compared with flat encodings applied to a 10^J -point grid, the proposed digit-wise constructions avoid the $O(10^J)$ growth of one-hot, unary, and domain-wall encodings and the $O(10^{J/2})$ growth of arithmetic-progression encodings, while remaining only slightly larger than flat binary. The main advantage is therefore not asymptotic bit optimality over binary, but a decimal-local structure that is naturally compatible with rolling-precision refinement and with our direct construction for selected nonlinear terms. For completeness, we further summarize how mixed-integer quadratic and quadratically constrained programs, as well as geometric programs with monomial terms, can be converted to QUBO using standard slack-variable, penalty, and auxiliary-variable constructions.

- We introduce a *rolling-precision* solution strategy that avoids solving a fully refined QUBO in one monolithic step. We develop two variants: (i) a *sequential bit-growth* scheme that increases discretization depth over selected variables, and (ii) a *constant-size zoom-in* scheme that keeps the QUBO size fixed and refines precision by adjusting variable bounds around incumbent solutions. Both variants incorporate backtracking to reject unproductive refinements, and the zoom-in variant can use dynamically scaled penalties to encourage constraint enforcement across shrinking domains.

The remainder of the paper is organized as follows. Section 2 introduces the problem class and the proposed encoding schemes for integer and continuous variables. Section 3 describes how mixed-integer quadratic and quadratically constrained programs, as well as geometric programs and selected nonlinear terms, are reformulated as QUBO. Section 4 presents the rolling-precision algorithm together with its theoretical properties. Section 5 reports computational results on illustrative examples. Finally, Section 6 summarizes the main findings and outlines directions for future research.

2 Problem Class and Encoding of Variables

First, we consider deriving a QUBO that approximates a general mixed-integer quadratically constrained quadratic program (MIQCQP) of the following form:

$$\begin{aligned}
& \min_{\mathbf{x}} \quad \mathbf{x}^\top \mathbf{H} \mathbf{x} + \mathbf{c}^\top \mathbf{x} \\
& \text{s.t.} \quad \mathbf{x}^\top \mathbf{C}_k \mathbf{x} + \mathbf{a}_k^\top \mathbf{x} \leq b_k, \quad \forall k \in \mathcal{K}, \\
& \quad \mathbf{x}^L \leq \mathbf{x} \leq \mathbf{x}^U, \\
& \quad \mathbf{x} \in \mathbb{R}^M \times \mathbb{Z}^P,
\end{aligned} \tag{2}$$

where $\mathbf{x} \in \mathbb{R}^{M+P}$ includes M continuous and P integer variables, $\mathbf{H} \in \mathbb{R}^{(M+P) \times (M+P)}$ is symmetric, $\mathbf{c} \in \mathbb{R}^{M+P}$, and for each $k \in \mathcal{K}$, we have $\mathbf{C}_k \in \mathbb{R}^{(M+P) \times (M+P)}$, $\mathbf{a}_k \in \mathbb{R}^{M+P}$, and $b_k \in \mathbb{R}$. We assume the bound vectors $\mathbf{x}^L$ and $\mathbf{x}^U$ are finite and known. By partitioning $\mathbf{x} = (\mathbf{x}^c, \mathbf{x}^i)$ into its continuous and integer components, we can write the Hessian $\mathbf{H}$ in block form as

$$\mathbf{H} = \begin{bmatrix} \mathbf{H}_{cc} & \mathbf{H}_{ci} \\ \mathbf{H}_{ci}^\top & \mathbf{H}_{ii} \end{bmatrix}, \tag{3}$$

where $\mathbf{H}_{cc} \in \mathbb{R}^{M \times M}$ contains purely continuous quadratic terms, $\mathbf{H}_{ii} \in \mathbb{R}^{P \times P}$ contains purely integer quadratic terms, and $\mathbf{H}_{ci} \in \mathbb{R}^{M \times P}$ couples continuous and integer variables. Several important subclasses of Eq. (2) can be recovered. For example, if $\mathbf{C}_k = 0$ for all $k \in \mathcal{K}$, we obtain a mixed-integer quadratic program (MIQP) with linear constraints:

$$\begin{aligned}
& \min_{\mathbf{x}} \quad \mathbf{x}^\top \mathbf{H} \mathbf{x} + \mathbf{c}^\top \mathbf{x} \\
& \text{s.t.} \quad \mathbf{A} \mathbf{x} \leq \mathbf{b}, \\
& \quad \mathbf{x} \in \mathbb{R}^M \times \mathbb{Z}^P,
\end{aligned} \tag{4}$$

for some $\mathbf{A} \in \mathbb{R}^{|\mathcal{K}| \times (M+P)}$ and $\mathbf{b} \in \mathbb{R}^{|\mathcal{K}|}$. If $M = 0$, Eq. (4) yields a purely integer quadratic program (IQP). If $P = 0$, Eq. (4) reduces to a quadratic program (QP) with linear constraints. If additionally $\mathbf{H} = 0$, we obtain a linear program (LP).

2.1 Encoding of integer variables

Let x_m be an integer variable with known bounds, i.e., $x_m \in \{L_m, L_m + 1, \dots, U_m\} \subset \mathbb{Z}$. We can represent x_m using a standard binary expansion. Define $R_m = U_m - L_m$ and choose the smallest nonnegative integer B_m such that $2^{B_m} - 1 \geq R_m$. Introduce binary variables $z_{m,0}, \dots, z_{m,B_m-1} \in \{0, 1\}$, and set

$$x_m = L_m + \sum_{b=0}^{B_m-1} 2^b z_{m,b}. \quad (5)$$

Any integer in the admissible range can be represented exactly by an appropriate choice of the binary vector $(z_{m,0}, \dots, z_{m,B_m-1})$, and the number of bits required is $B_m = \lceil \log_2(R_m + 1) \rceil$.

Illustrative Example 1.

Consider an integer variable $x \in \{3, 4, \dots, 10\}$. Here $L = 3$, $U = 10$, and $R = 7$, so we choose $B = 3$ because $2^3 - 1 = 7$. The encoding

$$x = 3 + z_0 + 2z_1 + 4z_2$$

with $z_0, z_1, z_2 \in \{0, 1\}$ represents all values from 3 (when $z = (0, 0, 0)$) up to 10 (when $z = (1, 1, 1)$).

2.2 Encoding of bounded continuous variables

Let x_m be a continuous variable with known bounds

$$x_m^L \leq x_m \leq x_m^U.$$

We first scale x_m to the unit interval by defining

$$\tilde{x}_m = \frac{x_m - x_m^L}{x_m^U - x_m^L} \in [0, 1], \quad x_m = x_m^L + (x_m^U - x_m^L) \tilde{x}_m. \quad (6)$$

We then approximate $\tilde{x}_m$ using a finite decimal expansion with J digits as described below. We show two different decimal encoding families. The first is a digit-sum unary encoding (Eq. 8), which is helpful for explaining the decimal-grid structure and for the one-digit nonlinear-term construction in Section 2.3. The second is a compact digit-wise specific binary encoding (SBE) (Eqs. 9–11), which is preferred when managing bit count is the main concern. We adopt decimal precision because the rolling-precision framework developed later refines resolution one decimal digit at a time, and because decimal places are readily interpretable in many engineering applications. However, this is not the only choice, as other strategies, e.g., binary-radix encodings, can use fewer bits for the

same absolute resolution, and alternative constructions (e.g., domain-wall encodings) offer different interaction structures and hardware tradeoffs.

2.2.1 Unary encoding

Consider a unary encoding that treats each admissible value as a point on a uniform decimal grid. Then the scaled continuous variable can *exactly* be represented as follows:

$$\tilde{x}_m = \sum_{j=1}^{\infty} \sum_{i=1}^{I=9} 10^{-j} z_{ijm} + 10^{-J} z_m, \quad (7)$$

where $z_{ijm} \in \{0, 1\}$. Any $\tilde{x}_m$ can be approximated upto J decimal places as follows:

$$\tilde{x}_m \approx \sum_{j=1}^J \sum_{i=1}^{I=9} 10^{-j} z_{ijm} + 10^{-J} z_m. \quad (8)$$

The term $10^{-J} z_m$ with $z_m \in \{0, 1\}$ is an endpoint bit that makes the upper bound (i.e., 1.000) representable on the scaled domain. Indeed, $\sum_{j=1}^J 9 \cdot 10^{-j} = 1 - 10^{-J}$. Therefore, without the endpoint bit, the maximum representable value would be $1 - 10^{-J}$. This formulation is conceptually straightforward but uses $9J + 1$ binary variables per continuous variable, which becomes prohibitive for large J , the number of binary variables increases by a factor of 9 for each decimal. Also, this formulation is generally non-unique, since multiple binary patterns can yield the same digit count. Therefore, we do not use it as a bit-efficient general-purpose encoding; rather, we use it to highlight the digit-wise grid structure of the encoding schemes.

Illustrative Example 2.

Suppose we wish to encode the decimal value $\tilde{x}_m = 0.342$ using $J = 3$ digits of precision. We write $\tilde{x}_m$ as the sum of its tenths, hundredths, and thousandths contributions:

$$\tilde{x}_m = 10^{-1} \sum_{i=1}^9 z_{i1m} + 10^{-2} \sum_{i=1}^9 z_{i2m} + 10^{-3} \sum_{i=1}^9 z_{i3m},$$

where $z_{ijm} \in \{0, 1\}$ are binary variables. To obtain $\tilde{x}_m = 0.342$, we enforce that exactly 3 of the variables in $\{z_{i1m}\}_{i=1}^9$ equal 1 (and the rest are 0), exactly 4 of the variables in

$\{z_{i2m}\}_{i=1}^9$ equal 1, and exactly 2 of the variables in $\{z_{i3m}\}_{i=1}^9$ equal 1. This yields

$$10^{-1}(3) + 10^{-2}(4) + 10^{-3}(2) = 0.3 + 0.04 + 0.002 = 0.342.$$

2.2.2 Digit-wise encoding with specific binary representation

To reduce the number of bits, we encode each decimal digit of $\tilde{x}_m$ separately. Formally, we write

$$\tilde{x}_m \approx \sum_{j=1}^J 10^{-j} d_{m,j} + 10^{-J} z_m, \quad (9)$$

where each $d_{m,j} \in \{0, 1, \dots, 9\}$ is the digit in position j , and $z_m \in \{0, 1\}$ is introduced to cover the offset and ensure that $\tilde{x}_m \in [0, 1]$. We then represent each digit $d_{m,j}$ using a small number of binary variables. One convenient choice is a weighted encoding with four bits per digit. Introduce binary variables $z_{m,j,1}, \dots, z_{m,j,4} \in \{0, 1\}$ and define

$$d_{m,j} = z_{m,j,1} + 2z_{m,j,2} + 3z_{m,j,3} + 3z_{m,j,4}. \quad (10)$$

Then the maximum possible digit value is $1 + 2 + 3 + 3 = 9$. Altogether, we obtain

$$\tilde{x}_m \approx \sum_{j=1}^J 10^{-j} \left(z_{m,j,1} + 2z_{m,j,2} + 3z_{m,j,3} + 3z_{m,j,4} \right) + 10^{-J} z_m. \quad (11)$$

Corollary 1: *Grid resolution.* For any $\tilde{x}_m \in [0, 1]$, there exists $\hat{x}_m$ such that $|\tilde{x}_m - \hat{x}_m| \leq 10^{-J}$ (since the grid spacing is $\approx 10^{-J}$). Then, upon scaling back, the maximum approximation error for a single variable is bounded by $(x_m^U - x_m^L) \cdot 10^{-J}$. Thus, for an n dimensional variable $\mathbf{x}$, the approximation error can be bounded as $\|\mathbf{x} - \hat{\mathbf{x}}\|_2 \leq \sqrt{n} \max_m (x_m^U - x_m^L) \cdot 10^{-J}$.

Illustrative Example 3.

Consider a variable $x \in [2, 5]$ and suppose we want $J = 3$ decimal digits. We first map x to $\tilde{x} \in [0, 1]$ via

$$\tilde{x} = \frac{x - 2}{3}.$$

Assume we wish to represent $\tilde{x} \approx 0.245$. With three digits, we have

$$\tilde{x} \approx 10^{-1}d_1 + 10^{-2}d_2 + 10^{-3}d_3 + 10^{-3}z,$$

with digits $d_1 = 2, d_2 = 4, d_3 = 5$. Using (10), one valid assignment is

$$d_1 = 2 \Rightarrow (z_{1,1}, z_{1,2}, z_{1,3}, z_{1,4}) = (0, 1, 0, 0),$$

$$d_2 = 4 \Rightarrow (z_{2,1}, z_{2,2}, z_{2,3}, z_{2,4}) = (1, 0, 1, 0),$$

$$d_3 = 5 \Rightarrow (z_{3,1}, z_{3,2}, z_{3,3}, z_{3,4}) = (0, 1, 1, 0).$$

These assignments yield

$$\tilde{x} \approx 0.2 + 0.04 + 0.005 = 0.245, \quad \text{and} \quad x \approx 2 + 3 \cdot 0.245 = 2.735.$$

Thus, we obtain an approximation of x with three decimal digits using $4J + 1 = 13$ binary variables. Figure 1 provides an overview of the unary and specific binary representation of a continuous variable.

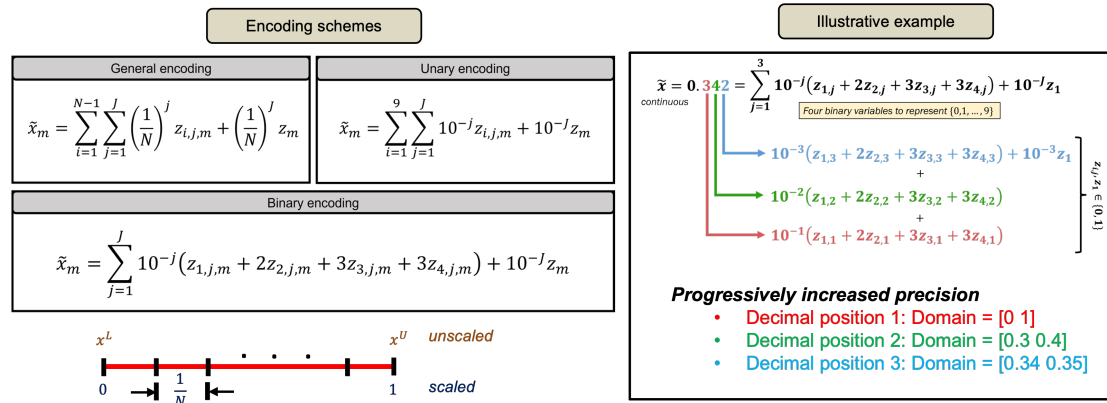


Figure 1: Encoding schemes to represent continuous variables as a collection of binary variables.

2.3 Encoding nonlinear terms

In the digit-sum unary encoding (Eq. 8), each decimal digit is represented by a sum of unary bits, and the digit value is the count of active bits. For the nonlinear-term construction below, we use a cumulative-unary version of the digit encoding rather than the nonunique digit-sum form in Eq. 8. In this representation, admissible states satisfy $z_{i+1} \leq z_i$. Therefore, a digit having a value of k is represented uniquely by $z_1 = \dots = z_k = 1$ and $z_{k+1} = \dots = z_9 = 0$. Since QUBO does not allow explicit ordering constraints, we enforce this structure using the equivalent penalty that can be added to the objective as follows: $\lambda_{nf} \sum_{j=1}^8 (z_{j+1} - z_{j+1} \cdot z_j)$, where λ_{nf} is a penalty parameter, and the penalty function is zero exactly when every active bit is followed only by active bits, i.e., when the pattern is cumulative unary. Next, we illustrate the encoding of certain nonlinear terms directly in terms of the binary variables. We consider scalar nonlinear terms of the form:

$$\psi(\tilde{x}) = \tilde{x}^a, \quad \tilde{x} \in [0, 1], \quad a \in \mathbb{R}, \quad (12)$$

where $\tilde{x}$ is a scaled continuous variable encoded via unary encoding as in Eq. (8). We assume that $\tilde{x}$ is represented with J decimal digits using the digit-wise encoding:

$$\tilde{x} = \sum_{j=1}^J (10^{-j} d_j) + 10^{-J} z, \quad d_j = \sum_{i=1}^9 z_{j,i},$$

where the $z, z_{j,i} \in \{0, 1\}$. In addition, we enforce a monotonicity constraints, i.e., $z \leq z_{j,9} \leq z_{j,8} \leq z_{j,7} \leq z_{j,6} \leq z_{j,5} \leq z_{j,4} \leq z_{j,3} \leq z_{j,2} \leq z_{j,1}$. For a fixed exponent a , we can approximate $\tilde{x}^a$ by a linear function of the same binary variables,

$$\tilde{x}^a \approx \sum_{j=1}^J \sum_{i=1}^9 P_{j,i} z_{j,i} + Qz, \quad (13)$$

where the coefficients $P_{j,i}$ and Q are precomputed. Note that, for a single decimal digit ($J = 1$) with a cumulative unary digit encoding, the nonlinear term $\psi(\tilde{x}) = \tilde{x}^a$ can be represented exactly as a linear function of the cumulative bits by precomputing the coefficients and summing them (see the example on the one-digit construction below). For multi-digit representations ($J > 1$), however, representing $\tilde{x}^a$ as a linear function of digit-wise binary variables is, in general, an approximation. This is because the number of adjustable linear

coefficients is $O(J)$, whereas the encoded grid contains $O(10^J)$ points. We therefore restrict any "exact" claim to the one-digit cumulative-unary construction, and treat the $J > 1$ construction as a grid-based surrogate whose approximation error may be quantified offline on the encoded grid. Specifically, for each exponent a and for each digit depth J , one can enumerate the admissible encoded grid $\mathcal{G}_J = \{0, 10^{-J}, 2 \cdot 10^{-J}, \dots, 1\}$ and compute the maximum absolute error and root-mean-square error (RMSE) of the surrogate relative to the exact function $\psi(\tilde{x}) = \tilde{x}^a$. For detailed analysis and error quantification, see Appendix A.1.

The grid-based surrogates used here can be viewed as a discretized approximation of $\psi(\tilde{x})$ on a bounded domain. Such approximations can also be implemented as piecewise linear functions using breakpoint/segment selection (e.g., SOS2). In a QUBO setting, an analogous selection structure can be expressed using binary variables together with quadratic penalties and (optionally) adjacency logic. The cumulative-unary digit encoding presented here can be viewed as a special ordered structure that naturally represents ordered digit states and is convenient for one-digit monotone constructions, particularly due to the existence of an equivalent penalty function.

One-digit construction

Suppose that $J = 1$, so that $\tilde{x} = 0.1d$ with $d \in \{0, \dots, 9\}$. Let $\gamma_i = 0.1i$ denote the grid points and $\varphi_i = \gamma_i^a$ the exact values of $\tilde{x}^a$ on this grid. We can enforce a cumulative encoding of the digit using binary variables $z_1, \dots, z_9$ with

$$d = \sum_{i=1}^9 z_i, \quad z_i \in \{0, 1\}, \quad z_{i+1} \leq z_i,$$

so that $d = k$ corresponds to $z_1 = \dots = z_k = 1$ and $z_{k+1} = \dots = z_9 = z = 0$. Now, by defining the coefficients

$$\Delta_i = \varphi_i - \varphi_{i-1}, \quad \varphi_0 = 0,$$

we can write

$$\tilde{x}^a = \gamma_d^a = \varphi_d = \sum_{i=1}^9 (\Delta_i z_i) + (1 - \varphi_9)z,$$

because the monotonicity pattern ensures that $\sum_{i=1}^9 \Delta_i z_i = \varphi_k$ whenever exactly the first k bits are equal to 1. Thus we obtain an exact representation of $\tilde{x}^a$ on the one-digit grid using a linear expression in the binary variables.

Illustrative Example 4.

Take $a = \frac{3}{2}$ and $J = 1$, so $\tilde{x} \in \{0, 0.1, \dots, 0.9, 1\}$. The grid values are $\gamma_i = 0.1i$ and $\varphi_i = \gamma_i^{3/2}$ for $i = 0, \dots, 9$. We define

$$\Delta_i = \gamma_i^{3/2} - \gamma_{i-1}^{3/2},$$

and represent

$$\tilde{x}^{3/2} = \sum_{i=1}^9 \Delta_i z_i + (1 - \varphi_9)z.$$

If $d = 4$ (i.e., $\tilde{x} = 0.4$), then $z_1 = z_2 = z_3 = z_4 = 1$ and $z_5 = \dots = z_9 = z = 0$, so

$$\tilde{x}^{3/2} = \sum_{i=1}^4 \Delta_i = \gamma_4^{3/2} = 0.4^{3/2},$$

as desired.

3 Reformulating Mixed-Integer Problems as QUBO

In this section, we show how the mixed-integer problems can be reformulated as QUBO using the encoding schemes presented in Section 2. Throughout Section 3, inequality constraints are converted to equalities with nonnegative slack variables and are then enforced through squared-residual penalties. Equality constraints are enforced directly through squared-residual penalties. After encoding all variables in binary form, these penalties yield an unconstrained binary quadratic objective suitable for QUBO solvers. We begin with mixed-integer quadratic programs with linear constraints, then treat quadratic constraints, geometric programs, and selected nonlinear terms.

3.1 Mixed-integer quadratic programs with linear constraints

We first consider the MIQP (4):

$$\begin{aligned} \min_{\mathbf{x}} \quad & \mathbf{x}^\top \mathbf{H} \mathbf{x} + \mathbf{c}^\top \mathbf{x} \\ \text{s.t.} \quad & \mathbf{A} \mathbf{x} \leq \mathbf{b}, \\ & \mathbf{x} \in \mathbb{R}^M \times \mathbb{Z}^P, \end{aligned} \tag{14}$$

We assume that bounds $x_m^L \leq x_m \leq x_m^U$ are known for all continuous variables and $x_p^L \leq x_p \leq x_p^U$ for all integer variables. For simplicity, we will assume that x_m is scaled, i.e., $x_m^L = 0$ and $x_m^U = 1$. These bounds are used in the encodings of Section 2 and in computing conservative bounds on slack variables.

3.1.1 Slack variables and penalty formulation

Each linear inequality constraint

$$\mathbf{a}_u^\top \mathbf{x} \leq b_u, \quad u = 1, \dots, U,$$

can be written as an equality constraint by introducing a nonnegative slack $s_u \geq 0$ variable:

$$\mathbf{a}_u^\top \mathbf{x} + s_u = b_u, \quad s_u \geq 0.$$

To encode each slack in binary form, we also require a finite upper bound $s_u^{\max}$. A conservative upper bound for s_u can be obtained by minimizing $\mathbf{a}_u^\top \mathbf{x}$ over the variable bounds. Specifically, we define

$$\phi_u = \min_{\mathbf{x} \in [\mathbf{x}^L, \mathbf{x}^U]} \mathbf{a}_u^\top \mathbf{x} = \sum_{m=1}^{M+P} \begin{cases} a_{u,m} x_m^L, & a_{u,m} \geq 0, \\ a_{u,m} x_m^U, & a_{u,m} < 0, \end{cases} \tag{15}$$

and set $s_u^{\max} = \max\{0, b_u - \phi_u\}$. Then, for every feasible point of the original constraints $\mathbf{a}_u^\top \mathbf{x} \leq b_u$ there exists $s_u \in [0, s_u^{\max}]$ such that $\mathbf{a}_u^\top \mathbf{x} + s_u = b_u$.

We collect the slacks into a vector $\mathbf{s} \in \mathbb{R}^U$ and express the inequities equivalently in compact form as $\mathbf{A} \mathbf{x} + \mathbf{s} = \mathbf{b}$, $\mathbf{s} \geq 0$. To fit the QUBO framework, we remove the explicit equality and inequality constraints by introducing a quadratic penalty term. For a penalty parameter $\lambda > 0$, we approximate the constrained MIQP by the unconstrained problem:

$$\min_{\mathbf{x}, \mathbf{s}} F(\mathbf{x}, \mathbf{s}) = \mathbf{x}^\top \mathbf{H} \mathbf{x} + \mathbf{c}^\top \mathbf{x} + \lambda \sum_{u=1}^U (\mathbf{a}_u^\top \mathbf{x} + s_u - b_u)^2, \quad (16)$$

where we restrict s_u to a bounded interval $[0, s_u^{\max}]$ and later encode both $\mathbf{x}$ and $\mathbf{s}$ by binary variables. For sufficiently large λ and appropriate bounds $s_u^{\max}$, the minimizer of (16) coincides with a solution of the original MIQP. We note, however, that the penalty parameter, λ must be chosen carefully. If λ is too small, the QUBO minimizer may violate the original constraints. If λ is too large, the elements of the resulting QUBO matrix can have a large dynamic range. This is especially important for quantum annealing hardware, where coefficients must be scaled to finite programmable ranges and are affected by analog control errors. Thus, squared penalties provide a convenient reformulation, but they can introduce numerical and hardware-sensitivity issues. In this work, penalty weights are treated as heuristic tuning parameters, and feasibility is assessed by reporting the resulting constraint residuals.

3.1.2 Binary encoding and QUBO construction

We next encode all decision variables by binary variables. Specifically, integer variables x_p are encoded using the binary expansion Eq. (5), and slack variables s_u are scaled to $[0, 1]$ via $s_u = s_u^{\max} \tilde{s}_u$ and encoded similarly to continuous variables. Let $\mathbf{z}$ denote the concatenated vector of all binary variables associated with $(\mathbf{x}, \mathbf{s})$, namely

$$\mathbf{z} = (\mathbf{z}^{(x)}, \mathbf{z}^{(s)}) \in \{0, 1\}^\beta,$$

where β is the total number of bits. Substituting the binary encodings into the penalized objective (16) yields a quadratic polynomial in $\mathbf{z}$:

$$F(\mathbf{x}, \mathbf{s}) = F(\mathbf{z}) = \mathbf{z}^\top \mathbf{Q} \mathbf{z} + \text{constant},$$

for some symmetric matrix $\mathbf{Q} \in \mathbb{R}^{\beta \times \beta}$. Note that, after substitution of the binary encodings, the penalized objective is a quadratic polynomial in $\mathbf{z}$ that may include linear terms $\mathbf{l}^\top \mathbf{z}$ and a constant. Because $z_i^2 = z_i$ for binary variables, linear terms can be absorbed into the diagonal of $\mathbf{Q}$. The constant term can be omitted from the optimization. Thus, the MIQP is reformulated as QUBO:

$$\min_{\mathbf{z} \in \{0,1\}^\beta} \mathbf{z}^\top \mathbf{Q} \mathbf{z}. \quad (17)$$

Let M be the number of continuous variables, P the number of integer variables, and U the number of slack variables introduced for linear inequalities. The total binary count is $\beta = \beta_x^{(c)} + \beta_x^{(i)} + \beta_s$. For digit-sum unary encoding, $\beta_x^{(c)} = M(9J + 1)$, and $\beta_s = U(9J + 1)$. For SBE, $\beta_x^{(c)} = M(4J + 1)$, and $\beta_s = U(4J + 1)$. Integer variables contribute $\beta_x^{(i)} = \sum_{p=1}^P B_p$, where $B_p = \lceil \log_2(x_p^U - x_p^L + 1) \rceil$, and $B_p = 1$ if x_p is binary.

Explicit expressions for the blocks can be derived by straightforward algebra and are provided in Appendix A.2 for completeness.

Illustrative Example 5.

Consider a simple constrained quadratic program

$$\min_{x \in [0,2]} (x - 1)^2 \quad \text{s.t.} \quad x \leq 1.5.$$

We introduce a slack $s \geq 0$ such that $x + s = 1.5$ and form the penalized objective

$$F(x, s) = (x - 1)^2 + \lambda(x + s - 1.5)^2,$$

with $s \in [0, 1.5]$. After scaling x and s to $[0, 1]$ and encoding both variables with, e.g., $J = 2$ decimal digits using (11), we obtain a QUBO in 18 binary variables (9 for x and 9 for s). Solving this QUBO yields an approximation of $x^* = 1$, $s^* = 0.5$, the solution of the original constrained problem.

3.2 Quadratic constraints via auxiliary variables

We now consider the MIQCQP (2). For clarity, we first treat the simpler case with only continuous variables and a single quadratic constraint,

$$\begin{aligned} \min_{\mathbf{x} \in \mathcal{X}} \quad & \mathbf{x}^\top \mathbf{H} \mathbf{x} + \mathbf{c}^\top \mathbf{x} \\ \text{s.t.} \quad & \mathbf{x}^\top \mathbf{C} \mathbf{x} + \mathbf{a}^\top \mathbf{x} \leq b, \end{aligned} \quad (18)$$

where $\mathcal{X} = \{\mathbf{x} \in \mathbb{R}^M : x_m^L \leq x_m \leq x_m^U\}$.

After scaling and encoding the continuous variables as in Section 2, each component x_m is represented as a quadratic form in its binary digits $\{z_{m,j,i}\}_{i=1}^4$ for each precision $j \in J$. Substituting these encodings into the quadratic constraint $\mathbf{x}^\top \mathbf{C} \mathbf{x} + \mathbf{a}^\top \mathbf{x} \leq b$ yields an inequality constraint in terms of binary variables that involves many bilinear products. We define the set of unique bilinear indices as $\mathcal{I} = \{(p, q) : 1 \leq p < q \leq N_z, \mu_{pq} \neq 0\}$, where μ_{pq} denotes the coefficient of the bilinear term $z_p z_q$ in the expanded constraint. Terms with $p = q$ satisfy $z_p^2 = z_p$ for binary variables and are therefore absorbed into the linear coefficients. No auxiliary variable is introduced for self-products. Therefore, a bilinear term can be expressed as $z_p z_q$, where $(p, q) \in \mathcal{I}$, and N_z is the number of bits used to represent $\mathbf{x}$. Note that $N_z = M(4J + 1)$ for SBE. For each $(p, q) \in \mathcal{I}$ we introduce a new binary variable $w_{pq} = z_p z_q$, where $w_{pq} \in \{0, 1\}$. The quadratic constraint can then be written as

$$\sum_{(p,q) \in \mathcal{I}} \mu_{pq} w_{pq} + \sum_{r=1}^{N_z} \nu_r z_r \leq b, \quad (19)$$

where the coefficients μ_{pq} and ν_r are determined by collecting like terms in $\mathbf{x}^\top \mathbf{C} \mathbf{x} + \mathbf{a}^\top \mathbf{x}$ after substitution of the encodings. Note that, the dense worst-case cardinality is $|\mathcal{I}| \leq \binom{N_z}{2} = \frac{N_z(N_z-1)}{2}$.

To enforce the relationship $w_{pq} = z_p z_q$ using only quadratic terms, we add an exact penalty function for each triple (z_p, z_q, w_{pq}) . A convenient choice is

$$\phi(z_p, z_q, w_{pq}) = 3w_{pq} - 2z_p w_{pq} - 2z_q w_{pq} + z_p z_q, \quad (20)$$

which satisfies $\phi(z_p, z_q, w_{pq}) = 0$ if and only if $w_{pq} = z_p z_q$, and is strictly positive otherwise (this can be verified by enumerating all eight combinations of $(z_p, z_q, w_{pq}) \in \{0, 1\}^3$). The penalty $\phi(z_p, z_q, w_{pq})$ is exact for the bilinear identity in the sense that $\phi = 0$ if and only if $w_{pq} = z_p z_q$. However, ensuring that a global minimizer of the full penalized objective satisfies all such identities (and satisfies the original inequality after slack penalization) requires penalty weights sufficiently large relative to competing objective terms. For implementation, a conservative penalty guideline can be obtained from finite-domain exact-penalty analysis (García et al. (2022), Verma & Lewis (2022)), which is given in Appendix A.3.

Incorporating the penalty in Eq. 20 with weight $P > 0$ into the objective yields

$$\begin{aligned}
& \min_{\mathbf{z}, \mathbf{w}} \quad \tilde{F}(\mathbf{z}) + P \sum_{(p,q) \in \mathcal{I}} \phi(z_p, z_q, w_{pq}) \\
& \text{s.t.} \quad \sum_{(p,q) \in \mathcal{I}} \mu_{pq} w_{pq} + \sum_{r=1}^{N_z} \nu_r z_r \leq b, \\
& \quad \mathbf{z} \in \{0, 1\}^{N_z}, \mathbf{w} \in \{0, 1\}^{|\mathcal{I}|},
\end{aligned} \tag{21}$$

where $\tilde{F}(\mathbf{z})$ is the QUBO representation of the quadratic objective $\mathbf{x}^\top \mathbf{H} \mathbf{x} + \mathbf{c}^\top \mathbf{x}$ after encoding $\mathbf{x}$ in terms of $\mathbf{z}$. As in the linear-constraint case, we can absorb the inequality constraint into the objective using a slack variable and a quadratic penalty, ultimately obtaining a pure QUBO formulation in $(\mathbf{z}, \mathbf{w})$:

$$\min_{(\mathbf{z}, \mathbf{w}) \in \{0, 1\}^{\gamma + |\mathcal{I}|}} \begin{bmatrix} \mathbf{z} \\ \mathbf{w} \end{bmatrix}^\top \mathbf{Q} \begin{bmatrix} \mathbf{z} \\ \mathbf{w} \end{bmatrix}, \tag{22}$$

where $\gamma = (M + U)(4J + 1)$ for SBE. The same construction extends to multiple quadratic constraints and to mixed-integer variables by following a similar technique.

Illustrative Example 6.

As a simple illustration, consider

$$\min_{x_1, x_2 \in [0, 1]} x_1^2 + x_2^2 \quad \text{s.t.} \quad x_1 x_2 \leq 0.2.$$

Using a single decimal digit ($J = 1$) for each variable, we encode

$$x_1 \approx 0.1 d_1, \quad x_2 \approx 0.1 d_2,$$

with digits $d_1, d_2 \in \{0, \dots, 9\}$ represented using four bits each (see Eq. (10)). The constraint $x_1 x_2 \leq 0.2$ becomes

$$0.01 d_1 d_2 \leq 0.2 \quad \Leftrightarrow \quad d_1 d_2 \leq 20.$$

The product $d_1 d_2$ expands into bilinear terms in the binary digits of d_1 and d_2 . Each such bilinear term is then replaced by an auxiliary variable w_{pq} with the penalty (Eq. (20)), and the resulting inequality is handled as in the MIQP case, leading to a QUBO in the

combined binary vector $(\mathbf{z}, \mathbf{w})$.

3.3 Geometric programs in monomial form

We next consider geometric programs in monomial form,

$$\begin{aligned}
\min_{\mathbf{x} \in \mathbb{R}_+^n} \quad & f_0(\mathbf{x}) = c_0 \prod_{\ell=1}^n x_\ell^{\alpha_\ell} \\
\text{s.t.} \quad & f_i(\mathbf{x}) = c_i \prod_{\ell=1}^n x_\ell^{\alpha_{i\ell}} \leq 1, \quad i = 1, \dots, m, \\
& h_j(\mathbf{x}) = c_j \prod_{\ell=1}^n x_\ell^{\alpha_{j\ell}} = 1, \quad j = 1, \dots, p, \\
& \mathbf{c} \in \mathbb{R}_+^{m+p+1}, \quad \boldsymbol{\alpha} \in \mathbb{R}^{n+nm+np}.
\end{aligned} \tag{23}$$

It is well known that geometric programs can be converted to convex problems by applying a logarithmic transformation (Boyd et al. (2007)). Let $z_\ell = \ln x_\ell$, $\ell = 1, \dots, n$, so that $x_\ell = e^{z_\ell}$. Taking logs of the objective and constraints in (23) yields an equivalent problem

$$\begin{aligned}
\min_{\mathbf{z} \in \mathbb{R}^n} \quad & f'_0(\mathbf{z}) = \ln c_0 + \sum_{\ell=1}^n \alpha_\ell z_\ell \\
\text{s.t.} \quad & f'_i(\mathbf{z}) = \ln c_i + \sum_{\ell=1}^n \alpha_{i\ell} z_\ell \leq 0, \quad i = 1, \dots, m, \\
& h'_j(\mathbf{z}) = \ln c_j + \sum_{\ell=1}^n \alpha_{j\ell} z_\ell = 0, \quad j = 1, \dots, p.
\end{aligned} \tag{24}$$

Thus the log-transformed geometric program is a linear program in the variables $\mathbf{z}$. If x_ℓ is bounded and strictly positive, i.e., $x_\ell \in [x_\ell^L, x_\ell^U]$ with $0 < x_\ell^L \leq x_\ell^U$, then under the log transform $x_\ell = \exp(z_\ell)$ we obtain explicit bounds $z_\ell \in [\ln x_\ell^L, \ln x_\ell^U]$. These bounds are then used by the encoding step. We can then treat Eq. (24) as a special case of the MIQP (Eq. (4)) with $\mathbf{H} = 0$ and continuous variables only. The techniques in Section 3.1 then yield a QUBO formulation for the geometric program, after encoding each z_ℓ via Eq. (11) and introducing appropriate slacks for the linear inequalities and equalities. Also, since $\arg \min_x f \equiv \arg \min_x \log f$, minimizing the objective function in Eq. (24) is equivalent to minimizing the objective function in Eq. (23).

3.4 Extension and other nonlinearities

Beyond quadratic and geometric structures, ideas presented in Section 2.3 can be applied to functions such as $\exp(\tilde{x})$, $\log(\tilde{x})$, $\sin(\tilde{x})$, or composite terms like $\prod_m \tilde{x}_m^{a_m}$. For multi-digit encodings ($J > 1$), similar constructions can be applied by working on a finer grid and precomputing function values at admissible points. In practice, one may approximate $\psi(\tilde{x})$ (see Eq. (12)) piecewise or via low-degree polynomial regressors and then rewrite these approximations in terms of binary digits. Composite terms such as $\prod_m \tilde{x}_m^{a_m}$ can be handled by encoding

$$\prod_m \tilde{x}_m^{a_m} = \exp\left(\sum_m a_m \ln \tilde{x}_m\right),$$

provided that $\tilde{x}_m$ stays away from zero so that $\ln \tilde{x}_m$ is well defined. This opens the door to QUBO reformulations of selected mixed-integer nonlinear programs (MINLPs) with signomial structure, albeit at the cost of additional auxiliary variables and larger QUBO sizes.

Note that the reformulation steps in this section are not specific to the digit-wise encodings. While the choice of encoding affects the resulting QUBO size, coefficient structure, and compatibility with the rolling-precision refinement strategy (described next), the reformulation strategies are encoding-agnostic and generally apply to any binary encoding of the decision and slack variables.

4 Rolling-Precision Algorithm

As discussed in Section 3, encoding continuous variables with high decimal precision and representing constraints via penalties can lead to extremely large QUBO instances, especially for MIQCQPs and MINLPs. Current quantum annealers and many classical QUBO solvers cannot handle such instances directly. To address this, we describe two different variants of a *rolling-precision* scheme illustrated in Figure 2 that solves a sequence of smaller QUBOs defined on coarser discretization grids and progressively refines the precision of selected variables: (i) sequential bit-growth scheme, and (ii) constant-size zoom-in scheme.

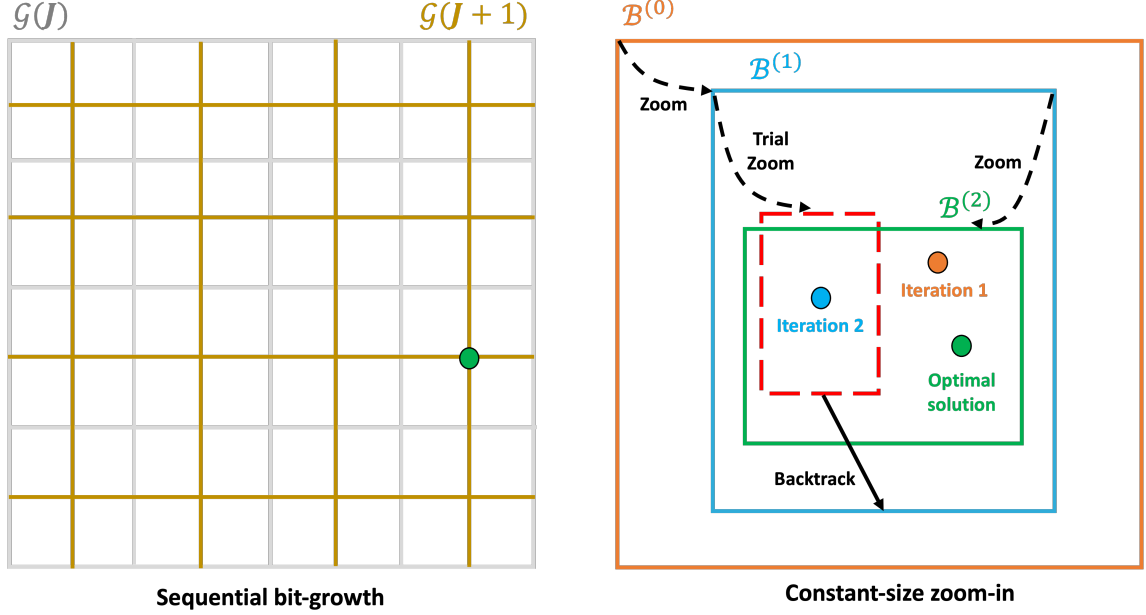


Figure 2: An illustration of the rolling-precision strategies. The sequential bit-growth is shown on the left, where a $[0, 1]^n$ is represented by a grid with precision J . Note that $\mathcal{G}(J)$ is not able to express the optimal solution (denoted by green dot). A further refinement of the grid is therefore necessary. This leads to a denser representation of the domain by $\mathcal{G}(J+1)$. Note that the grids are nested, i.e., $\mathcal{G}(J) \subseteq \mathcal{G}(J+1)$. The constant-size zoom-in scheme is shown on the right. In the event of aggressive shrinking or unsatisfactory improvement of the objective, a backtracking is triggered and the algorithm resorts to a previously accepted box. Through this step, the QUBO size remains unchanged, and only the variable bounds are rescaled based on the incumbent solution.

4.1 Sequential Bit-Growth Scheme

This strategy involves progressively increasing J , i.e., adding more bits to allow for finer solutions. Although the bit-growth variant increases QUBO size when additional digits are introduced, it can still be useful for three reasons. First, some variables may stabilize at low precision, allowing early termination without ever constructing the largest monolithic QUBO (consider at the optimal solution $x_1 = 0.12$ and $x_2 = 0.3456$. In this case, two decimal precision is sufficient for x_1 while x_2 requires four decimal precision). If the solution stabilizes at $J < J_{max}$, one can stop early and never build the larger monolithic QUBO. Second, refinement can be applied selectively to a subset of variables (and, when applicable,

to a subset of slack variables) rather than uniformly to every encoded quantity. Third, in practical heuristic settings, solving a sequence of smaller QUBOs can be more reliable than solving one large, high-precision QUBO immediately. However, this is a practical motivation rather than a worst-case complexity guarantee, since in the worst case, all variables may still be refined to $\mathbf{J}_{\max}$.

Grids and precisions.

Let $F : [0, 1]^n \rightarrow \mathbb{R}$ denote the objective function of the original continuous problem after scaling all variables to $[0, 1]$. For each variable x_i scaled to $[0, 1]$, a decimal precision $J_i \in \mathbb{Z}_{\geq 0}$ corresponds to a uniform grid

$$\mathcal{G}_i(J_i) := \{\ell 10^{-J_i} : \ell = 0, 1, \dots, 10^{J_i}\} \subset [0, 1]. \quad (25)$$

This grid contains exactly the numbers in $[0, 1]$ that can be written with J_i decimal places and therefore matches the digit-wise encodings in Section 2. Refining the precision from J_i to $J_i + 1$ refines the grid by a factor of 10 and preserves all previous points, i.e., $\mathcal{G}_i(J_i) \subseteq \mathcal{G}_i(J_i + 1)$. For a precision vector $\mathbf{J} = (J_1, \dots, J_n)$, we define the product grid

$$\mathcal{G}(\mathbf{J}) := \prod_{i=1}^n \mathcal{G}_i(J_i) \subseteq [0, 1]^n. \quad (26)$$

We assume the following:

- A1** (*ε -suboptimality*) For any precision vector $\mathbf{J}$, the QUBO solver returns a solution $\mathbf{x}^*(\mathbf{J}) \in \mathcal{G}(\mathbf{J})$ such that $F(\mathbf{x}^*(\mathbf{J})) \leq F^*(\mathbf{J}) + \varepsilon$, where $F^*(\mathbf{J}) = \min_{\mathbf{x} \in \mathcal{G}(\mathbf{J})} F(\mathbf{x})$ and $\varepsilon \geq 0$ is a solver-dependent tolerance.
- A2** (*Lower boundedness*) The function F is bounded below on $[0, 1]^n$, i.e., there exists $F_{\inf}$ with $F(\mathbf{x}) \geq F_{\inf}$ for all $\mathbf{x}$.
- A3** (*Nested grids*) The refinement property $\mathcal{G}_i(J_i + 1) \supseteq \mathcal{G}_i(J_i)$ holds for all i and all J_i .

Sequential bit-growth without backtracking.

We first present a forward-only rolling-precision scheme, which is the version we analyze theoretically. We maintain the following: (i) a current precision vector $\mathbf{J}$, (ii) an incumbent

solution $\mathbf{x}^* \in \mathcal{G}(\mathbf{J})$, and (iii) its objective value $F^* = F(\mathbf{x}^*)$. We also specify a maximum precision $J_{\max,i}$ for each variable and an improvement threshold $\theta > 0$. At each iteration, we scan each variable and attempt to refine its precision by one decimal digit (see Algorithm 1). Note that, we denote by $\text{QuboSolve}(F, \mathcal{G}(\mathbf{J}))$ the solution returned by the QUBO solver on the grid $\mathcal{G}(\mathbf{J})$.

Algorithm 1 Sequential bit-growth without backtracking

Require: Objective F , initial precision $\mathbf{J}_0 \in \mathbb{Z}_{\geq 0}^n$, maximum precision $\mathbf{J}_{\max} \in \mathbb{Z}_{\geq 0}^n$, improvement threshold $\theta > 0$

```

1:  $\mathbf{J} \leftarrow \mathbf{J}_0$ 
2:  $\mathbf{x}^* \leftarrow \text{QuboSolve}(F, \mathcal{G}(\mathbf{J}))$ 
3:  $F^* \leftarrow F(\mathbf{x}^*)$ 
4: repeat
5:    $improved \leftarrow \text{false}$ 
6:   for  $i = 1, \dots, n$  do ▷ Try to refine variable  $x_i$ 
7:     if  $J_i = J_{\max,i}$  then continue
8:     end if
9:      $\mathbf{J}^{\text{trial}} \leftarrow \mathbf{J}; \quad J_i^{\text{trial}} \leftarrow J_i + 1$ 
10:     $\mathbf{x}^{\text{trial}} \leftarrow \text{QuboSolve}(F, \mathcal{G}(\mathbf{J}^{\text{trial}}))$ 
11:     $F^{\text{trial}} \leftarrow F(\mathbf{x}^{\text{trial}})$ 
12:    if  $F^{\text{trial}} \leq F^* - \theta$  then
13:       $\mathbf{J} \leftarrow \mathbf{J}^{\text{trial}}$ 
14:       $\mathbf{x}^* \leftarrow \mathbf{x}^{\text{trial}}, F^* \leftarrow F^{\text{trial}}$ 
15:       $improved \leftarrow \text{true}$ 
16:    end if
17:  end for
18: until  $improved = \text{false}$  or  $\mathbf{J} = \mathbf{J}_{\max}$ 
19: return  $(\mathbf{x}^*, F^*, \mathbf{J})$ 

```

We first note that each *accepted* refinement strictly decreases F^* and that the algorithm terminates after finitely many trials.

Lemma 1: *Monotone incumbents. Under Assumptions A1–A3, whenever Algorithm 1 accepts a refinement, the incumbent objective strictly decreases by at least θ . Between accepts, F^* remains constant.*

Proof. Whenever a refinement for variable x_i is accepted, the acceptance criterion (line 12 of Algorithm 1) requires $F^{\text{trial}} \leq F^* - \theta$. The incumbent is then updated to $F_{\text{new}}^* = F^{\text{trial}}$

(line 14 in Algorithm 1), giving

$$F_{\text{new}}^* = F^{\text{trial}} \leq F_{\text{old}}^* - \theta < F_{\text{old}}^*$$

where the strict inequality uses $\theta > 0$. Thus, the sequence of accepted incumbents is strictly decreasing. Between accepts, F^* is not updated and hence constant. Since F is bounded below by F_{inf} on $[0, 1]^n$ by Assumption **A2**, we have $F^* \geq F_{\text{inf}}$ at all times. $\square$

Assumption **A1** permits the QUBO solver to return an ε -suboptimal solution on the grid $\mathcal{G}(\mathbf{J})$. The acceptance criterion is applied to the observed function value $F^{\text{trial}} = F(\mathbf{x}^{\text{trial}})$, which is evaluated exactly at the returned binary point. Consequently, the decrease guarantee of θ holds for the observed incumbent sequence regardless of ε . In order to ensure that an accepted refinement represents true progress towards the true grid optimum, the threshold should satisfy $\theta > \varepsilon$.

Lemma 2: *Finite number of accepted refinements. The number of accepted refinements in Algorithm 1 is finite and bounded by $\sum_{i=1}^n (J_{\text{max},i} - J_{0,i})$.*

Proof. Each accepted refinement increases some J_i by one. No component of $\mathbf{J}$ can exceed $J_{\text{max},i}$, so each J_i can be incremented at most $(J_{\text{max},i} - J_{0,i})$ times. Hence, the total number of accepted refinements is at most $\sum_i (J_{\text{max},i} - J_{0,i})$. $\square$

Lemma 3: *Finite work between accepts. In each outer iteration of Algorithm 1, at most n trial QUBO solves are performed.*

Proof. Each outer loop executes the for-loop over $i = 1, \dots, n$ once and attempts at most one refinement per variable in that loop. $\square$

The following result highlights two basic properties of the bit-growth scheme under our global-subproblem assumptions, i.e., finite termination over a finite set of refinement states and monotone non-increase of the incumbent objective across accepted refinements. We do not claim a stronger convergence guarantee beyond the discretized grids considered.

Theorem 1 (Finite termination and monotone objective (no backtracking)). *Under Assumptions **A1–A3**, Algorithm 1 terminates after finitely many trial refinements. The incumbent values F^* form a strictly decreasing sequence across accepted refinements and never increase throughout the algorithm.*

Proof. Combining Lemmas 2 and 3 shows that only finitely many trial refinements are possible. Monotonicity follows from Lemma 1. $\square$

If Algorithm 1 terminates before reaching the maximum precision $\mathbf{J}_{\max}$, then the final solution is locally optimal with respect to the grid refinements considered. To illustrate this, suppose Algorithm 1 terminates with $\mathbf{J}^* \neq \mathbf{J}_{\max}$. Then for every variable i with $J_i^* < J_{\max,i}$, refining its precision by one digit (i.e., considering $\mathbf{J}^{\text{trial}} = \mathbf{J}^* + e_i$) does not yield any improvement larger than θ , i.e., $F(\mathbf{x}^*(\mathbf{J}^{\text{trial}})) > F^* - \theta$, where $\mathbf{x}^*(\mathbf{J}^{\text{trial}})$ is the QUBO-solver returned minimizer of F on $\mathcal{G}(\mathbf{J}^{\text{trial}})$. Because, at termination, the algorithm has completed a full sweep over $i = 1, \dots, n$ with *improved* = **false**. For each i with $J_i^* < J_{\max,i}$, the corresponding trial precision $\mathbf{J}^{\text{trial}} = \mathbf{J}^* + e_i$ was considered, and the associated trial solution did not satisfy $F^{\text{trial}} > F^* - \theta$. Otherwise, the refinement would have been accepted.

Sequential bit-growth with backtracking.

In practice Assumption **A1** may not hold. Therefore, it can be beneficial to allow *backtracking*, i.e., to occasionally reduce the precision of some variables or to revert to a previously accepted state if a refinement turns out not to be promising when combined with refinements of other variables. Here we outline a simple backtracking variant that preserves monotonic improvement of the incumbent objective and still terminates in finitely many steps. The key ideas are as follows:

- Maintain a stack of previously accepted states $(\mathbf{J}, \mathbf{x}^*, F^*)$,
- Allow both *refinement moves* $\mathbf{J} \rightarrow \mathbf{J} + e_i$ and *backtracking moves* $\mathbf{J} \rightarrow \mathbf{J} - e_i$ (provided $J_i > J_{0,i}$),
- Accept any move (refinement or backtrack) only if it yields a strict decrease of the objective by at least θ ,
- Keep track of visited precision vectors $\mathbf{J}$ to avoid cycling.

Algorithm 2 Rolling-precision QUBO with backtracking

Require: $F, \mathbf{J}_0, \mathbf{J}_{\max}, \theta > 0$

```
1:  $\mathbf{J} \leftarrow \mathbf{J}_0$ 
2:  $\mathbf{x}^* \leftarrow \text{QuboSolve}(F, \mathcal{G}(\mathbf{J}))$ 
3:  $F^* \leftarrow F(\mathbf{x}^*)$ 
4:  $\mathcal{V} \leftarrow \{\mathbf{J}\}$  ▷ Visited precision vectors
5:  $\mathcal{S} \leftarrow \{(\mathbf{J}, \mathbf{x}^*, F^*)\}$  ▷ Stack of accepted states
6: repeat
7:    $improved \leftarrow \text{false}$ 
8:   for each candidate move  $\mathbf{J} \rightarrow \mathbf{J}'$  (refinement or backtrack) with  $\mathbf{J}' \notin \mathcal{V}$  do
9:     Solve QUBO on  $\mathcal{G}(\mathbf{J}')$  to obtain  $\mathbf{x}'$  and  $F' = F(\mathbf{x}')$ 
10:    if  $F' \leq F^* - \theta$  then
11:       $\mathbf{J} \leftarrow \mathbf{J}', \mathbf{x}^* \leftarrow \mathbf{x}', F^* \leftarrow F'$ 
12:       $\mathcal{V} \leftarrow \mathcal{V} \cup \{\mathbf{J}\}$ 
13:      Push  $(\mathbf{J}, \mathbf{x}^*, F^*)$  onto  $\mathcal{S}$ 
14:       $improved \leftarrow \text{true}$ 
15:    end if
16:  end for
17: until  $improved = \text{false}$  or no unvisited  $\mathbf{J}'$  remain
18: return  $(\mathbf{x}^*, F^*, \mathbf{J})$ 
```

In this backtracking variant, the incumbent objective F^* remains strictly decreasing because we only accept moves (either refinements or backtracks) that improve the objective by at least θ . To avoid cycles in the precision space, we maintain the visited set $\mathcal{V}$ and do not revisit any precision vector $\mathbf{J}$ once it has been examined. Since the set $\{\mathbf{J} \in \mathbb{Z}^n : J_{0,i} \leq J_i \leq J_{\max,i}\}$ is finite, and each visited $\mathbf{J}$ is never considered again, the algorithm terminates after finitely many steps. A detailed complexity analysis for specific move-selection policies is beyond the scope of this paper, but experimentally, we find that backtracking may help escape poor refinement paths while preserving the monotone-improvement property of the incumbent objective.

One could argue that, since Algorithms 1 and 2 end up increasing $\mathbf{J}$ whenever a solution is improved, in the worst case, one would reach $\mathbf{J}_{\max}$, thereby ultimately recreating the monolithic problem. To that end, we present the second variant of the rolling-precision algorithm, where we fix the QUBO size and adjust the variable bounds to progressively arrive at a good solution.

4.2 Constant-size rolling-precision via zoom-in boxes

As described above, the sequential bit-growth rolling-precision scheme increases decimal precision by appending new digit bits, which enlarges the QUBO at each refinement step. In many applications, however, the primary goal of “increasing precision” is not to represent the entire $[0, 1]^n$ domain more finely, but rather to *refine the search locally* around an incumbent solution. This motivates a constant-size rolling-precision scheme where the key idea is to keep the encoding depth, $\mathbf{J}$ fixed (fixed number of binary variables per continuous/slack variable) and instead *shrink the variable bounds* around the incumbent solution, thereby increasing effective resolution on a progressively refined grid without increasing QUBO size.

Boxes, grids, and constant QUBO size.

Let us fix an encoding depth $J \in \mathbb{Z}_{\geq 0}$ for each continuous variable. These depths are held *constant* throughout the algorithm. At iteration t , we maintain an axis-aligned search box

$$\mathcal{B}^{(t)} := \prod_{i=1}^n [L_i^{(t)}, U_i^{(t)}], \quad 0 \leq L_i^{(t)} \leq U_i^{(t)} \leq 1. \quad (27)$$

Given $\mathcal{B}^{(t)}$, the fixed-depth digit-wise encoding in Section 2 (applied to each coordinate after affine scaling from $[0, 1]$ to $[L_i^{(t)}, U_i^{(t)}]$) induces a uniform grid on each coordinate:

$$\mathcal{G}_i^{(t)}(J) := \left\{ L_i^{(t)} + (U_i^{(t)} - L_i^{(t)})\tilde{x}_i \right\} \subset [L_i^{(t)}, U_i^{(t)}]. \quad (28)$$

where, $\tilde{x}_i$ is represented using Eq. 9 or Eq. 10. The corresponding product grid is

$$\mathcal{G}^{(t)} := \prod_{i=1}^n \mathcal{G}_i^{(t)}(J) \subset \mathcal{B}^{(t)}. \quad (29)$$

Crucially, the number of binary variables per continuous variable is fixed by J , so the QUBO dimension remains constant across all iterations; only the QUBO *coefficients* change with the current box $\mathcal{B}^{(t)}$.

Zoom operator.

Let $\rho_t \in (0, 1]$ be a shrink factor. Given an incumbent $\mathbf{x}_t^* \in \mathcal{B}^{(t)}$, we define the next box $\mathcal{B}^{(t+1)} = \text{Zoom}(\mathcal{B}^{(t)}, \mathbf{x}_t^*)$ by shrinking/updating each coordinate interval and centering around $\mathbf{x}_t^*$:

$$\begin{aligned} W_i^{(t)} &:= U_i^{(t)} - L_i^{(t)}, & W_i^{(t+1)} &:= \rho_t W_i^{(t)}, \\ L_i^{(t+1)} &:= \max\{0, x_{t,i}^* - \tfrac{1}{2}W_i^{(t+1)}\}, & U_i^{(t+1)} &:= \min\{1, x_{t,i}^* + \tfrac{1}{2}W_i^{(t+1)}\}. \end{aligned} \quad (30)$$

Other centering/anchoring rules are possible; however, the key property is that interval widths are adjusted dynamically while remaining within $[0, 1]$ along each dimension.

Illustrative zoom-in example.

Consider a single continuous variable $x \in [0, 1]$ and fix a constant encoding depth $J = 1$. Assuming a decimal representation, the grid on *any* interval $[L, U]$ is

$$\mathcal{G}(J = 1) = \{L + (U - L)\ell 10^{-1} : \ell = 0, 1, \dots, 10\}.$$

Starting from the full domain $\mathcal{B}^{(0)} = [0, 1]$, the grid points are $\{0, 0.1, 0.2, \dots, 1.0\}$. Suppose the QUBO solve on this coarse grid returns an incumbent $x^* = 0.4$. Choose a shrink factor $\rho = 0.2$. Then the next box is centered at x^* with width 0.2:

$$\mathcal{B}^{(1)} = \left[x^* - \tfrac{1}{2}\rho, x^* + \tfrac{1}{2}\rho\right] = [0.3, 0.5].$$

Importantly, the encoding depth is unchanged ($J = 1$), so the QUBO size does not change; however, the new grid is now

$$\mathcal{G}(J = 1) = \{0.3 + 0.2\ell 10^{-1} : \ell = 0, 1, \dots, 10\} = \{0.30, 0.32, 0.34, \dots, 0.50\},$$

which corresponds to an *effective resolution* of 0.02 in the original $[0, 1]$ domain despite still using only $J = 1$ digits, showing that the search becomes progressively finer while the number of binary variables remains constant.

4.2.1 Zoom-in rolling-precision algorithm without backtracking

We analyze the scheme under the same idealized assumptions as in the bit-growth variant, but now applied to box-restricted grids. We first present a forward-only zoom-in scheme, and maintain the following: (i) a current box $\mathcal{B}$, (ii) an incumbent solution $\mathbf{x}^* \in \mathcal{G}(\mathcal{B})$, and (iii) its objective value $F^* = F(\mathbf{x}^*)$. We also specify a minimum box width $W_{\min}$ and an improvement threshold $\theta > 0$. At each iteration, we attempt a zoom step and accept it only if it yields a strict improvement of at least θ .

Algorithm 3 Constant-size zoom-in rolling-precision QUBO without backtracking

Require: Objective F , initial box $\mathcal{B}^{(0)}$, fixed encoding depth (J) , shrink factor $\rho \in (0, 1]$, minimum width $W_{\min}$, improvement threshold $\theta > 0$

```

1:  $\mathcal{B} \leftarrow \mathcal{B}^{(0)}$ 
2:  $\mathbf{x}^* \leftarrow \text{QuboSolve}(F, \mathcal{G}(\mathcal{B}))$ 
3:  $F^* \leftarrow F(\mathbf{x}^*)$ 
4: repeat
5:    $\mathcal{B}^{\text{trial}} \leftarrow \text{Zoom}(\mathcal{B}, \mathbf{x}^*)$ 
6:    $\mathbf{x}^{\text{trial}} \leftarrow \text{QuboSolve}(F, \mathcal{G}(\mathcal{B}^{\text{trial}}))$ 
7:    $F^{\text{trial}} \leftarrow F(\mathbf{x}^{\text{trial}})$ 
8:   if  $F^{\text{trial}} \leq F^* - \theta$  then
9:      $\mathcal{B} \leftarrow \mathcal{B}^{\text{trial}}$ 
10:     $\mathbf{x}^* \leftarrow \mathbf{x}^{\text{trial}}, F^* \leftarrow F^{\text{trial}}$ 
11:   else
12:     break
13:   end if
14: until  $\min_i (U_i - L_i) \leq W_{\min}$ 
15: return  $(\mathbf{x}^*, F^*, \mathcal{B})$ 

```

Finite termination and monotonicity.

Each accepted zoom step strictly decreases the incumbent objective. Moreover, the box widths shrink geometrically, so only finitely many accepted zooms are possible before the minimum widths are reached.

Lemma 1 (Zoom monotonicity): *Under Assumptions A1–A3, whenever Algorithm 3 accepts a zoom step, the incumbent objective decreases by at least θ .*

Proof. The acceptance test requires $F^{\text{trial}} \leq F^* - \theta$. When accepted, the incumbent is updated to $F_{\text{new}}^* = F^{\text{trial}}$, hence $F_{\text{new}}^* \leq F_{\text{old}}^* - \theta < F_{\text{old}}^*$. Between accepts, F^* is

unchanged. □

Then, we can define the minimum resolvable change in the objective function at precision $\mathbf{J}$ as

$$\Delta_{\min}(\mathbf{J}) = \min_{\substack{\mathbf{x}, \tilde{\mathbf{x}} \in G(\mathbf{J}) \\ \mathbf{x} \neq \tilde{\mathbf{x}}}} |F(\mathbf{x}) - F(\tilde{\mathbf{x}})|.$$

We require $\theta > \Delta_{\min}(\mathbf{J})$ to ensure that accepted refinements correspond to distinct solution values.

Lemma 2 (Finite number of accepted zooms): *The number of accepted zoom steps is finite and bounded by*

$$\sum_{i=1}^n \left\lceil \frac{\log(W_{i,\min}/W_i^{(0)})}{\log(\rho)} \right\rceil_+,$$

where $W_i^{(0)} = U_i^{(0)} - L_i^{(0)}$ and $\lceil \cdot \rceil_+$ denotes $\max\{0, \lceil \cdot \rceil\}$.

Proof. Each accepted zoom reduces W_i by at least a factor ρ until the floor $W_{i,\min}$ is reached. Thus, for each i , there can be at most $\left\lceil \log(W_{i,\min}/W_i^{(0)}) / \log(\rho) \right\rceil_+$ accepted reductions. Summing over coordinates implies the bound. □

Theorem 2 (Finite termination and monotone objective (zoom-in, no backtracking)). *Under Assumptions A1–A3, Algorithm 3 terminates after finitely many trial zooms. The incumbent values F^* form a strictly decreasing sequence across accepted zoom steps and never increase throughout the algorithm.*

Proof. Monotonicity follows from Lemma 1 and lower boundedness from Assumption A2. Finite termination follows from Lemma 2 and the fact that each iteration performs only one trial zoom. □

Implementation note. The termination and monotonicity statements above apply to the idealized scalar-objective version of the zoom-in algorithm. For constrained numerical examples, we use a practical lexicographic merit rule: a candidate is accepted if it improves feasibility, or, once both incumbent and candidate are feasible within tolerance, if it improves the original objective. This implementation is intended to improve practical

feasibility behavior under finite penalties and should not be interpreted as a global optimality guarantee.

4.2.2 Zoom-in rolling-precision algorithm with backtracking

In practice, QUBO solvers are often heuristic and may return suboptimal solutions on some boxes. Aggressive shrinking can also prematurely exclude promising regions. We therefore consider a backtracking variant that reverts to previously accepted boxes when no improving zoom move is found.

- Maintain a stack of previously accepted states $(\mathcal{B}, \mathbf{x}^*, F^*)$,
- Allow candidate moves such as: (i) zoom all variables, (ii) zoom a subset of variables, (iii) temporarily *expand* a box (inverse zoom) if desired,
- Accept any candidate box only if it improves the incumbent by at least θ ,
- Maintain a visited set of boxes to avoid cycling.

Algorithm 4 Constant-size zoom-in rolling-precision QUBO with backtracking

Require: F , initial box $\mathcal{B}^{(0)}$, fixed depth (J), shrink factor ρ , minimum width $W_{\min}$, threshold $\theta > 0$

- 1: $\mathcal{B} \leftarrow \mathcal{B}^{(0)}$
- 2: $\mathbf{x}^* \leftarrow \text{QuboSolve}(F, \mathcal{G}(\mathcal{B}))$
- 3: $F^* \leftarrow F(\mathbf{x}^*)$
- 4: $\mathcal{V} \leftarrow \{\mathcal{B}\}$ ▷ Visited box
- 5: $\mathcal{S} \leftarrow \{(\mathcal{B}, \mathbf{x}^*, F^*)\}$ ▷ Stack of accepted states
- 6: **repeat**
- 7: $improved \leftarrow \text{false}$
- 8: **for** each candidate box $\mathcal{B}'$ generated from $(\mathcal{B}, \mathbf{x}^*)$ **do**
- 9: **if** $\mathcal{B}' \in \mathcal{V}$ **then continue**
- 10: **end if**
- 11: Solve QUBO on $\mathcal{G}(\mathcal{B}')$ to obtain $\mathbf{x}'$ and $F' = F(\mathbf{x}')$
- 12: $\mathcal{V} \leftarrow \mathcal{V} \cup \{\mathcal{B}'\}$
- 13: **if** $F' \leq F^* - \theta$ **then**
- 14: $\mathcal{B} \leftarrow \mathcal{B}', \mathbf{x}^* \leftarrow \mathbf{x}', F^* \leftarrow F'$
- 15: Push $(\mathcal{B}, \mathbf{x}^*, F^*)$ onto $\mathcal{S}$
- 16: $improved \leftarrow \text{true}$
- 17: **break**
- 18: **end if**
- 19: **end for**
- 20: **if** $improved = \text{false}$ **then** ▷ Backtrack
- 21: **if** $|\mathcal{S}| > 1$ **then**
- 22: Pop $\mathcal{S}$ (remove the current accepted state)
- 23: $(\mathcal{B}, \mathbf{x}^*, F^*) \leftarrow \text{top}(\mathcal{S})$ ▷ Revert to the previous accepted state
- 24: **else**
- 25: **break**
- 26: **end if**
- 27: **end if**
- 28: **until** $\min_i (U_i - L_i) \leq W_{\min}$
- 29: **return** $(\mathbf{x}^*, F^*, \mathcal{B})$

Dynamic penalty scaling.

When bounds shrink, the magnitude of equality/inequality residuals scales with the interval widths. To maintain consistent constraint enforcement across zoom levels, we scale penalty

weights as

$$\lambda^{(t)} = \min \left\{ \lambda_{\max}, \lambda_0 / (\omega^{(t)})^2 \right\}, \quad \omega^{(t)} = \max_i (U_i^{(t)} - L_i^{(t)}), \quad (31)$$

This ensures that penalties strengthen as the search region narrows.

Note that the backtracking variant mitigates, but does not eliminate, the risk of excluding promising regions. By reverting to previously accepted boxes and optionally allowing subset zooms or temporary box expansion, the method can recover from some overly aggressive local refinements. However, the candidate boxes are not exhaustive, and the procedure remains a local heuristic. Therefore, the backtracking scheme does not provide a guarantee of global optimality or solution quality.

5 Numerical Experiments

The purpose of this section is to illustrate the workflow of the proposed QUBO reformulation and rolling-precision strategies, rather than to provide a scalability or practical performance analysis. The examples are intentionally small so that the effects of discretization depth, finite penalties, feasibility residuals, and QUBO-size growth can be inspected directly. All numerical experiments are carried out on a MacBook Pro with an Apple M4 Pro chip (12-core CPU) and 24 GB of RAM, running macOS. The QUBO subproblems are solved using a simulated-annealing QUBO solver (D-Wave `neal`) with the following parameters: `num_reads` = 200, `num_sweeps` = 2500, `seed` = 11. The rolling-precision framework is solver-agnostic and can be paired with other heuristic or exact QUBO solvers when available.

5.1 Example 1: Unconstrained quadratic program

We first consider a simple two-dimensional unconstrained problem to illustrate how the rolling-precision scheme refines the solution:

$$\min_{x_1, x_2 \in [0,1]} f(x_1, x_2) = (x_1 - 0.1234567)^2 + (x_2 - 0.7654321)^2. \quad (32)$$

The unique minimizer is $x_1^* = 0.1234567$, $x_2^* = 0.7654321$, and $f(x^*) = 0$. We encode each variable using the specific binary digit-wise scheme in Section 2, with a decimal precision

vector $\mathbf{J} = (J_1, J_2)$. For each fixed $\mathbf{J}$, the corresponding QUBO is solved. In the bit-growth scheme, increasing decimal precision $\mathbf{J}$ adds digit-bits, so the QUBO size grows with $\mathbf{J}$. Table 1 shows that moving from $(J_1, J_2) = (2, 2)$ to $(8, 8)$ increases the QUBO from 18 to 66 binary variables and from 72 to 1056 quadratic terms, while reaching the global minimum (up to floating-point roundoff) exactly at $(8, 8)$. In the zoom-in scheme, the number of bits is fixed (here $J = 1$ for each variable), so the QUBO size stays constant. Higher accuracy is achieved by shrinking the bounds around the incumbent (“zooming in”). Table 2 shows that the QUBO remains at 10 binary variables and 20 quadratic terms across iterations, and the objective improves from 1.745×10^{-3} (no zoom) to 4.069×10^{-9} after four accepted zoom steps. Backtracking occurs once further zoom attempts fail to improve the incumbent.

Overall, bit-growth attains machine-level accuracy but at the cost of rapidly increasing QUBO size, whereas zoom-in keeps the QUBO size fixed and attains comparable accuracy with much fewer variables.

Table 1: Example 1 (bit-growth rolling-precision).

Iter	(J_1, J_2)	x_1	x_2	Obj	#vars	#quad
0	(2,2)	0.1200000	0.7700000	3.281×10^{-5}	18	72
1	(4,4)	0.1235000	0.7654000	2.905×10^{-9}	34	272
2	(6,6)	0.1234570	0.7654320	1.000×10^{-13}	50	600
3	(8,8)	0.1234567	0.7654321	4.950×10^{-32}	66	1056
Monolithic						
–	(8,8)	0.1234567	0.7654321	4.950×10^{-32}	66	1056

Table 2: Example 1 (constant-size zoom-in rolling-precision, $J = 1$ fixed).

Iter	Action	x_1	x_2	Obj	#vars	#quad
0	Baseline (no zoom)	0.100000	0.800000	1.745×10^{-3}	10	20
1	Accepted zoom	0.120000	0.760000	4.146×10^{-5}	10	20
2	Accepted zoom	0.124000	0.764000	2.346×10^{-6}	10	20
3	Accepted zoom	0.123200	0.765600	9.409×10^{-8}	10	20
4	Accepted zoom (best)	0.123520	0.765440	4.069×10^{-9}	10	20
Backtracking observed after Iter 4 when further zoom attempts did not improve the incumbent.						

5.2 Example 2: Mixed-integer quadratic program

We consider the mixed-integer quadratic program (MIQP)

$$\begin{aligned} \min_{x,y} \quad & (x - 0.35)^2 + 0.2y \\ \text{s.t.} \quad & x + 0.8y \leq 1, \\ & x \in [0, 1], \\ & y \in \{0, 1\}. \end{aligned}$$

We convert the inequality into an equality using a slack $s \in [0, 1]$:

$$x + 0.8y + s = 1, \quad s \geq 0,$$

and enforce it via a quadratic penalty $\lambda(x + 0.8y + s - 1)^2$ in the QUBO objective (here $\lambda = 100$). Variables x and s are encoded using SBE decimal bits, and y is a single binary variable. In the bit-growth strategy, as shown in Table 3, refining from (1, 1) to (2, 2) already identifies the exact optimum $(x, y, s) = (0.35, 0, 0.65)$ with essentially zero objective and negligible constraint residual. The monolithic solve at (4, 4) matches the same solution but at a larger QUBO size. In the constant-size zoom-in strategy, the number of bits is fixed (here $J = 1$), and accuracy is improved by shrinking bounds around the incumbent. Table 4 shows that the QUBO size remains constant ($\#vars = 11$, $\#quad = 55$) while the objective improves from 2.5×10^{-3} (no zoom) to 1.6×10^{-7} after accepted zoom steps, and essentially arriving at the same global optimal solution. Backtracking is triggered when further zoom attempts fail to yield improvement.

Table 3: Example 2 (bit-growth rolling-precision with backtracking, $\lambda = 100$).

Iter	(J_x, J_s)	x	y	s	Obj	$\max(0, x + 0.8y - 1)$	#vars	#quad
0	(1,1)	0.400000	0	0.600000	2.500×10^{-3}	0.0×10^0	11	55
1	(2,2)	0.350000	0	0.650000	1.233×10^{-32}	2.2×10^{-16}	19	171
Monolithic (max precision)								
–	(4,4)	0.350000	0	0.650000	1.233×10^{-32}	2.2×10^{-16}	35	595

Table 4: Example 2 (constant-size zoom-in rolling-precision, fixed $J = 1$, $\lambda = 100$).

Iter	Action	x	y	s	Obj	#vars	#quad
0	Baseline (no zoom)	0.400000	0	0.600000	2.500×10^{-3}	11	55
1	Accepted zoom	0.360000	0	0.640000	1.000×10^{-4}	11	55
2	Accepted zoom	0.352000	0	0.648000	4.000×10^{-6}	11	55
3	Backtrack	0.352000	0	0.648000	4.000×10^{-6}	11	55
4	Accepted zoom (best)	0.349600	0	0.650400	1.600×10^{-7}	11	55
Backtracking occurs after Iter 4 when zoom candidates fail to improve the incumbent.							

5.3 Example 3: MINLPLib instance Alan

We report our results on the MINLPLib **Alan** instance (<https://www.minlplib.org/alan.html>) as an illustrative demonstration of the QUBO construction and rolling-precision behavior under a heuristic QUBO solver. Because heuristic annealing does not certify global optimality, we avoid global-optimum claims and instead compare against the best-known feasible solution (MINLPLib reports a best-known solution with objective value 2.925). We also report constraint residuals for all enforced equalities/inequalities to investigate feasibility under finite penalties.

With continuous variables $x_1, \dots, x_4 \in [0, 1]$ and binaries $y_1, \dots, y_4 \in \{0, 1\}$, the problem instance is given below:

$$\begin{aligned}
\min \quad & 4x_1^2 + 6x_1x_2 - 2x_1x_3 + 6x_2^2 + 2x_2x_3 + 10x_3^2 \\
\text{s.t.} \quad & x_1 + x_2 + x_3 + x_4 = 1 \\
& 8x_1 + 9x_2 + 12x_3 + 7x_4 = 10 \\
& x_i \leq y_i, \quad i = 1, \dots, 4 \\
& y_1 + y_2 + y_3 + y_4 \leq 3.
\end{aligned} \tag{33}$$

Each inequality is rewritten using a nonnegative slack and enforced via a squared-penalty:

$$x_i - y_i + s_i = 0, \quad s_i \in [0, 1], \quad (y_1 + y_2 + y_3 + y_4) + s_c = 3, \quad s_c \in [0, 3],$$

and we add $\lambda_1(x_1 + x_2 + x_3 + x_4 - 1)^2 + \lambda_2(8x_1 + 9x_2 + 12x_3 + 7x_4 - 10)^2 + \lambda_3 \sum_i (x_i - y_i + s_i)^2 + \lambda_4(y_1 + y_2 + y_3 + y_4 + s_c - 3)^2$ to the QUBO objective. All continuous variables and slacks are encoded using SBE. We use the following penalty parameter values in the

rolling-precision experiments: $\lambda_1 = \lambda_2 = \lambda_3 = \lambda_4 = 500$ (penalty-sensitivity analysis is given in Appendix A.4). Note that these values are selected as a fixed heuristic for this illustrative experiment. We did not optimize this value, and it should not be interpreted as hardware-calibrated. Larger penalties generally improve constraint enforcement but also increase the coefficient dynamic range of the QUBO, which can make the instance more sensitive to coefficient scaling and hardware noise.

Because the QUBO formulation enforces constraints through finite penalty weights, objective values are not by themselves sufficient to establish equivalence with a feasible benchmark solution. Therefore, we report the original objective together with feasibility residuals. Specifically, for the equality constraints we report absolute residuals, while for inequality constraints we report positive violations. For reporting purposes, constraints are denoted as c_i in Table 5, i.e., the second equality constraint is denoted as c_2 and the Maximum violation is defined as $\max(\max_{i=1,\dots,4} \max(0, x_i - y_i), \max(0, \sum_{i=1}^4 y_i - 3))$. In the bit-growth strategy, Table 5 shows that early low-precision states can satisfy subsets of constraints well (e.g., the second constraint, c_2 is nearly exact at (1,1)), but as precision grows the method explores different discrete assignments and balances objective versus penalties. The monolithic solve at (8,8) yields a larger QUBO (301 variables; 16,044 quadratic terms) and longer runtime. In the constant-size zoom-in strategy, the number of bits is fixed (here $J = 1$), so the QUBO size remains constant. Accuracy is improved by shrinking bounds around the incumbent (zooming), with backtracking if a zoom attempt fails to improve. As shown in Table 6, a single accepted zoom step improves the objective from 3.000 to 2.928 while maintaining essentially zero feasibility error, all at a constant QUBO size (46 variables; 385 quadratic terms). Note that, here the Maximum violation is defined as $\max(|c_1|, |c_2|, \max_{i=1,\dots,4} \max(0, x_i - y_i), \max(0, \sum_{i=1}^4 y_i - 3))$.

Table 5: Example 3 (bit-growth rolling-precision with backtracking)

Iter	(J_x, J_s)	x_1, x_2, x_3, x_4	y_1, y_2, y_3, y_4	Obj	$ c_1 $	$ c_2 $	Maximum violation	#vars	#quad
0	(1,1)	[0.300,0.000,0.400,0.400]	[1,0,1,1]	1.7200	1.0×10^{-1}	1.8×10^{-15}	0.0×10^0	49	406
1	(2,2)	[0.340,0.330,0.360,0.000]	[1,1,1,0]	3.0778	3.0×10^{-2}	1.0×10^{-2}	0.0×10^0	85	1248
2	(3,3)	[0.297,0.264,0.435,0.004]	[1,1,1,0]	3.1050	0.0×10^0	1.8×10^{-15}	4.0×10^{-3}	121	2554
3	(4,4)	[0.498,0.087,0.434,0.0031]	[1,1,1,0]	2.824198	2.2×10^{-2}	3.3×10^{-3}	3.1×10^{-3}	157	4324
4	(5,4)	[0.315,0.0033,0.5333,0.150]	[1,0,1,1]	2.914832	1.6×10^{-3}	7.0×10^{-4}	3.3×10^{-3}	173	5820
Monolithic									
–	(8,8)	[0.3967,0.1813,0.4269,0.0100]	[1,1,1,0]	2.896760	1.5×10^{-2}	1.9×10^{-3}	1.0×10^{-2}	301	16044

Table 6: Example 3 (constant-size zoom-in rolling-precision with backtracking).

Step	Action	x_1, x_2, x_3, x_4	y_1, y_2, y_3, y_4	Obj	Maximum violation	#vars	#quad
0	Baseline (no zoom)	[0.500,0.000,0.500,0.000]	[1,0,1,1]	3.000000	0.00×10^0	46	385
1	Accepted zoom_xs	[0.400,0.000,0.520,0.080]	[1,0,1,1]	2.928000	1.78×10^{-15}	46	385

Backtracking occurs after Step 1 when zoom candidates fail to improve the incumbent.

6 Conclusions

Quantum optimization platforms, including quantum annealers and QUBO-based classical solvers, are naturally suited for binary quadratic models but cannot directly handle continuous variables, general integer domains, and explicit constraints. However, many problems in process systems engineering, energy systems, and related fields are formulated as mixed-integer programs with linear and nonlinear constraints. Bridging this gap requires systematic encoding and reformulation strategies that map such problems into QUBO while keeping problem sizes manageable. In this work, we have presented a unified mixed-integer-to-QUBO workflow that consolidates encoding and reformulation ingredients and combines them with rolling-precision strategies for managing QUBO growth as precision is refined. While some of the reformulation and penalty constructions used here are inspired by prior QUBO modeling practice, our contribution is in organizing these ingredients into an explicit end-to-end pipeline, presenting two new decimal-precision encoding choices within that pipeline, and studying two rolling-precision variants (bit-growth and zoom-in) that trade off QUBO size against encoded solution quality. First, we introduced encodings for bounded continuous and integer variables based on decimal digit-wise representations with controllable precision. These encodings can represent continuous variables and selected nonlinear expressions, such as power functions, using structured collections of binary variables. Second, we showed how mixed-integer quadratic programs, quadratically constrained quadratic programs, geometric programs in monomial form, and certain signomial terms can be reformulated as QUBO by combining these encodings with slack variables and exact quadratic penalty constructions for both linear and quadratic constraints. Third, recognizing that high-precision encodings can lead to prohibitively large QUBO instances, we developed a rolling-precision algorithm that solves a sequence of smaller QUBOs defined on nested discretization grids. At each step, the algorithm selectively increases the precision of individual variables only when it

yields sufficient objective improvement. Under mild assumptions on the QUBO solver, we established monotonic decrease in objective and finite termination property with respect to the considered refinements. Numerical examples confirmed that the rolling-precision scheme can recover high-accuracy solutions while keeping individual QUBO sizes moderate. Although the examples in this paper use base-10 digits, the rolling-precision framework only requires a nested sequence of discretization grids. Therefore, the same refinement idea applies to any radix ($B \geq 2$), including standard binary-radix encodings. Several directions for future work remain. On the modeling side, it would be valuable to explore automated selection of encodings and penalty parameters tailored to specific problem classes. On the algorithmic side, incorporating more sophisticated variable selection and adaptive precision schemes may further improve the tradeoff between solution quality and QUBO size. Finally, evaluating the proposed framework on realistic large-scale process systems engineering case studies and on emerging quantum annealing hardware would provide deeper insight into its practical potential and limitations. Larger process systems case studies would require additional problem-specific modeling, penalty calibration, solver tuning, and hardware-aware benchmarking, and are therefore left for future work.

Acknowledgement

M.M.F.H. gratefully acknowledges support from the Supplement for the NSF CAREER award CBET-1943479 and the ACS PRF New Directions Award 67330-ND9. Part of the research was conducted with the computing resources provided by Texas A&M High Performance Research Computing.

Code and Data Availability

The scripts used to generate the QUBO models and reproduce the numerical experiments are publicly available at <https://github.com/aiftakher/rolling-precision-qubo>.

References

Ajagekar, A., Al Hamoud, K. & You, F. (2022), ‘Hybrid classical-quantum optimization techniques for solving mixed-integer programming problems in production scheduling’, *IEEE Transactions on Quantum Engineering*.

- Ajagekar, A., Humble, T. & You, F. (2020), ‘Quantum computing based hybrid solution strategies for large-scale discrete-continuous optimization problems’, *Computers & Chemical Engineering* **132**, 106630.
- Albash, T. & Lidar, D. A. (2018), ‘Adiabatic quantum computation’, *Reviews of Modern Physics* **90**(1), 015002.
- Baldassi, C. & Zecchina, R. (2018), ‘Efficiency of quantum vs. classical annealing in nonconvex learning problems’, *Proceedings of the National Academy of Sciences* **115**(7), 1457–1462.
- Barahona, F. (1982), ‘On the computational complexity of ising spin glass models’, *Journal of Physics A: Mathematical and General* **15**(10), 3241.
- Bernal, D. E., Ajagekar, A., Harwood, S. M., Stober, S. T., Trenev, D. & You, F. (2022), ‘Perspectives of quantum computing for chemical engineering’, *AIChE Journal* **68**(6), e17651.
- Biegler, L. T., Grossmann, I. E. & Westerberg, A. W. (1997), ‘Systematic methods for chemical process design’.
- Booth, M., Reinhardt, S. P. & Roy, A. (2017), ‘Partitioning optimization problems for hybrid classical’, *quantum execution. Technical Report* pp. 01–09.
- Boothby, K., Bunyk, P., Raymond, J. & Roy, A. (2020), ‘Next-generation topology of d-wave quantum processors with pegasus’, *arXiv preprint arXiv:2003.00133*.
- Boros, E. & Hammer, P. L. (2002), ‘Pseudo-boolean optimization’, *Discrete applied mathematics* **123**(1-3), 155–225.
- Boyd, S., Kim, S.-J., Vandenberghe, L. & Hassibi, A. (2007), ‘A tutorial on geometric programming’, *Optimization and engineering* **8**(1), 67–127.
- Cai, J., Macready, W. G. & Roy, A. (2014), ‘A practical heuristic for finding graph minors’, *arXiv preprint arXiv:1406.2741*.
- Chancellor, N. (2019), ‘Domain wall encoding of discrete variables for quantum annealing and qaoa’, *Quantum Science and Technology* **4**(4), 045004.
- Chang, C.-Y., Jones, E., Yao, Y., Graf, P. & Jain, R. (2020), ‘On hybrid quantum and classical computing algorithms for mixed-integer programming’, *arXiv e-prints* pp. arXiv–2010.
- Coppersmith, D. (2002), ‘An approximate fourier transform useful in quantum factoring’, *arXiv preprint quant-ph/0201067*.
- Deutsch, D. & Jozsa, R. (1992), ‘Rapid solution of problems by quantum computation’, *Proceedings of the Royal Society of London. Series A: Mathematical and Physical Sciences* **439**(1907), 553–558.
- García, M. D., Ayodele, M. & Moraglio, A. (2022), Exact and sequential penalty weights in quadratic unconstrained binary optimisation with a digital annealer, in ‘Proceedings of the Genetic and Evolutionary Computation Conference Companion’, pp. 184–187.
- Glover, F., Kochenberger, G. & Du, Y. (2019), ‘Quantum bridge analytics i: a tutorial on formulating and using qubo models’, *4OR* **17**, 335–371.
- Grant, E. K. & Humble, T. S. (2020), Adiabatic quantum computing and quantum annealing, in ‘Oxford Research Encyclopedia of Physics’.
- Grover, L. K. (1996), A fast quantum mechanical algorithm for database search, in ‘Proceedings of the twenty-eighth annual ACM symposium on Theory of computing’, pp. 212–219.
- Hauke, P., Katzgraber, H. G., Lechner, W., Nishimori, H. & Oliver, W. D. (2020), ‘Perspectives of quantum annealing: Methods and implementations’, *Reports on Progress in Physics* **83**(5), 054401.

- Iftakher, A., Kazi, M.-K. & Hasana, M. F. (2023), ‘Mixed-integer quadratic optimization using quantum computing for process applications’, *Proceeding of the Foundations of Computer Aided Process Operations/Chemical Process Control* pp. 1–6.
- Iftakher, A., Leonard, T. & Hasan, M. M. F. (2025), ‘Integrating different fidelity models for process optimization: A case of equilibrium and rate-based extractive distillation using ionic liquids’, *Computers & Chemical Engineering* **192**, 108890.
- Iftakher, A., Monjur, M. S. & Hasan, M. F. (2023), ‘An overview of computer-aided molecular and process design’, *Chemie Ingenieur Technik* **95**(3), 315–333.
- Iftakhera, A. & Hasana, M. F. (2024), ‘Exploring quantum optimization for computer-aided molecular and process design’.
- Ishikawa, H. (2010), ‘Transformation of general binary mrf minimization to the first-order case’, *IEEE transactions on pattern analysis and machine intelligence* **33**(6), 1234–1249.
- Karimi, S. & Ronagh, P. (2019), ‘Practical integer-to-binary mapping for quantum annealers: S. karimi, p. ronagh’, *Quantum Information Processing* **18**(4), 94.
- Kochenberger, G., Hao, J.-K., Glover, F., Lewis, M., Lü, Z., Wang, H. & Wang, Y. (2014), ‘The unconstrained binary quadratic programming problem: a survey’, *Journal of combinatorial optimization* **28**(1), 58–81.
- Koska, O., Baboulin, M. & Gazda, A. (2025), ‘A mixed-precision quantum-classical algorithm for solving linear systems’.
URL: <https://arxiv.org/abs/2502.02212>
- Lanting, T., Przybysz, A. J., Smirnov, A. Y., Spedalieri, F. M., Amin, M. H., Berkley, A. J., Harris, R., Altomare, F., Boixo, S., Bunyk, P. et al. (2014), ‘Entanglement in a quantum annealing processor’, *Physical Review X* **4**(2), 021041.
- Lucas, A. (2014), ‘Ising formulations of many np problems’, *Frontiers in physics* **2**, 5.
- Lucas, A. (2019), ‘Hard combinatorial problems and minor embeddings on lattice graphs’, *Quantum Information Processing* **18**(7), 203.
- Mohammadisiahroudi, M., Augustino, B., Sampourmahani, P. & Terlaky, T. (2025), ‘Quantum computing inspired iterative refinement for semidefinite optimization’, *Mathematical Programming* pp. 1–40.
- Murali, P. et al. (2019), Noise-adaptive compiler mappings for noisy quantum computers, in ‘Proceedings of the Twenty-Fourth International Conference on Architectural Support for Programming Languages and Operating Systems’, pp. 1015–1029.
- Nielsen, M. A. & Chuang, I. (2002), ‘Quantum computation and quantum information’.
- Razavy, M. (2013), *Quantum theory of tunneling*, World Scientific.
- Shor, P. W. (1999), ‘Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer’, *SIAM review* **41**(2), 303–332.
- Tamura, K., Shirai, T., Katsura, H., Tanaka, S. & Togawa, N. (2021), ‘Performance comparison of typical binary-integer encodings in an ising machine’, *IEEE Access* **9**, 81032–81039.
- Tanaka, S., Tamura, R. & Chakrabarti, B. K. (2017), *Quantum spin glasses, annealing and computation*, Cambridge University Press.
- Valiante, E., Hernandez, M., Barzegar, A. & Katzgraber, H. G. (2021), ‘Computational overhead of locality reduction in binary optimization problems’, *Computer Physics Communications* **269**, 108102.

- Verma, A. & Lewis, M. (2022), ‘Penalty and partitioning techniques to improve performance of qubo solvers’, *Discrete Optimization* **44**, 100594.
- Xavier, P. M., Ripper, P., Andrade, T., Garcia, J. D., Maculan, N. & Neira, D. E. B. (2023), ‘Qubo. jl: A julia ecosystem for quadratic unconstrained binary optimization’, *arXiv preprint arXiv:2307.02577* .
- Yarkoni, S., Raponi, E., Bäck, T. & Schmitt, S. (2022), ‘Quantum annealing for industry applications: Introduction and review’, *Reports on Progress in Physics* .
- Zhao, Z., Fan, L. & Han, Z. (2022), Hybrid quantum benders’ decomposition for mixed-integer linear programming, *in* ‘2022 IEEE Wireless Communications and Networking Conference (WCNC)’, IEEE, pp. 2536–2540.

Appendix

A.1 Offline error quantification for the multi-digit nonlinear surrogate

We quantify the approximation error introduced by the multi-digit nonlinear surrogate described in Section 2.3. We consider scalar nonlinear terms of the form

$$\psi(\tilde{x}) = \tilde{x}^a, \quad \tilde{x} \in [0, 1], \quad (34)$$

where $\tilde{x}$ denotes the scaled continuous variable and $a > 0$ is a known exponent. For a fixed digit depth J , the admissible encoded grid is

$$\mathcal{G}_J = \{0, 10^{-J}, 2 \cdot 10^{-J}, \dots, 1\}. \quad (35)$$

For each grid point $\tilde{x} = k10^{-J}$, with $k = 0, 1, \dots, 10^J$, we write its decimal representation using J digits as

$$\tilde{x} = \sum_{j=1}^J 10^{-j} d_j + 10^{-J} z, \quad (36)$$

where $d_j \in \{0, 1, \dots, 9\}$ is the digit at decimal position j , and $z \in \{0, 1\}$ is the endpoint bit. For all non-endpoint grid points, $z = 0$. For the endpoint $\tilde{x} = 1$, we use $d_j = 9$ for all $j = 1, \dots, J$ and $z = 1$. Following the coefficient construction in Section 2.3, the digit-wise coefficients are

$$P_{j,i} = (i10^{-j})^a - ((i-1)10^{-j})^a, \quad j = 1, \dots, J, \quad i = 1, \dots, 9. \quad (37)$$

The endpoint coefficient is

$$Q = 1 - \sum_{j=1}^J \sum_{i=1}^9 P_{j,i}^{(a)}. \quad (38)$$

Therefore, the multi-digit surrogate is evaluated as

$$\hat{\psi}_{a,J}(\tilde{x}) = \sum_{j=1}^J \sum_{i=1}^{d_j} P_{j,i} + Qz. \quad (39)$$

For completeness, Table 7 reports the one-digit coefficients used in the cumulative-unary construction. These coefficients are useful for reproducing the illustrative nonlinear encoding examples and for verifying the exactness of the $J = 1$ representation. For $J > 1$, the coefficients are generated by the same formula in Eq. (37).

Table 7: One-digit cumulative-unary coefficients for the nonlinear surrogate $\psi(\tilde{x}) = \tilde{x}^a$. For $J = 1$, these coefficients give an exact representation on the grid $\{0, 0.1, \dots, 1\}$ up to numerical roundoff.

a	P_1	P_2	P_3	P_4	P_5	P_6	P_7	P_8	P_9	Q
0.6	0.251189	0.129542	0.104863	0.091487	0.082674	0.076268	0.071322	0.067345	0.064051	0.061260
1.5	0.031623	0.057820	0.074874	0.088665	0.100571	0.111205	0.120904	0.129880	0.138273	0.146185

To quantify the approximation quality, we define the pointwise surrogate error as: $e_{a,J}(\tilde{x}) = \psi(\tilde{x}) - \hat{\psi}_{a,J}(\tilde{x})$, and report two grid-level error metrics. The maximum absolute error is

$$E_{\max}(a, J) = \max_{\tilde{x} \in \mathcal{G}_J} |e_{a,J}(\tilde{x})|, \quad (40)$$

and the root-mean-square error is

$$\text{RMSE}(a, J) = \left(\frac{1}{|\mathcal{G}_J|} \sum_{\tilde{x} \in \mathcal{G}_J} e_{a,J}(\tilde{x})^2 \right)^{1/2}, \quad (41)$$

$|\mathcal{G}_J|$ denotes the number of grid points $= 10^J + 1$. Table 8 reports these errors for the exponents used in the nonlinear encoding examples.

Table 8: Offline approximation error of the nonlinear digit-wise surrogate $\hat{\psi}_{a,J}(\tilde{x})$ for $\psi(\tilde{x}) = \tilde{x}^a$ over the admissible encoded grid $\mathcal{G}_J$.

Exponent a	Digits J	Grid size $ \mathcal{G}_J $	Max. absolute error	RMSE
0.6	1	11	1.11×10^{-16}	3.74×10^{-17}
0.6	2	101	1.81×10^{-1}	1.09×10^{-1}
0.6	3	1001	2.34×10^{-1}	1.39×10^{-1}
1.5	1	11	1.11×10^{-16}	5.02×10^{-17}
1.5	2	101	1.04×10^{-1}	4.26×10^{-2}
1.5	3	1001	1.17×10^{-1}	4.68×10^{-2}

The results confirm that the one-digit cumulative-unary construction is exact on the

encoded grid up to numerical roundoff. However, for $J > 1$, the digit-wise linear surrogate can introduce non-negligible error. This behavior is expected because the multi-digit surrogate is additive across decimal positions, whereas the true nonlinear function $\tilde{x}^a$ is not additively separable in its decimal digits (may contain interacting terms). Consequently, the $J > 1$ construction should be viewed as a surrogate for the nonlinear encoding. In applications where the nonlinear surrogate affects feasibility or optimality, its error should be quantified over the encoded grid before use, or a higher-fidelity piecewise/grid-selection encoding should be adopted.

A.2 Construction of the QUBO matrix

We describe how the QUBO matrix in Eq. (17) is constructed from the penalized mixed-integer quadratic program in Section 3.1.

Penalized quadratic form in the original variables. Recall the MIQP with linear constraints in its expanded form:

$$\begin{aligned} \min_{\mathbf{x}, \mathbf{y}} \quad & \mathbf{x}^\top \mathbf{Q}_1 \mathbf{x} + \mathbf{y}^\top \mathbf{Q}_2 \mathbf{y} + \mathbf{x}^\top \mathbf{Q}_3 \mathbf{y} + \mathbf{c}^\top \mathbf{x} + \mathbf{d}^\top \mathbf{y} \\ \text{s.t.} \quad & \mathbf{A} \mathbf{x} + \mathbf{B} \mathbf{y} \leq \mathbf{e}, \\ & \mathbf{x} \in \mathbb{R}^M, \mathbf{y} \in \{0, 1\}^P, \end{aligned} \tag{42}$$

where $\mathbf{Q}_1 \in \mathbb{R}^{M \times M}$, $\mathbf{Q}_2 \in \mathbb{R}^{P \times P}$, $\mathbf{Q}_3 \in \mathbb{R}^{M \times P}$, $\mathbf{A} \in \mathbb{R}^{U \times M}$, and $\mathbf{B} \in \mathbb{R}^{U \times P}$.

We introduce a slack variable $\mathbf{s} \in \mathbb{R}_+^U$ such that

$$\mathbf{A} \mathbf{x} + \mathbf{B} \mathbf{y} + \mathbf{s} = \mathbf{e}, \quad \mathbf{s} \geq 0,$$

and form the quadratic penalty objective

$$F(\mathbf{x}, \mathbf{y}, \mathbf{s}) = \mathbf{x}^\top \mathbf{Q}_1 \mathbf{x} + \mathbf{y}^\top \mathbf{Q}_2 \mathbf{y} + \mathbf{x}^\top \mathbf{Q}_3 \mathbf{y} + \mathbf{c}^\top \mathbf{x} + \mathbf{d}^\top \mathbf{y} + \lambda \|\mathbf{A} \mathbf{x} + \mathbf{B} \mathbf{y} + \mathbf{s} - \mathbf{e}\|_2^2, \tag{43}$$

with $\lambda > 0$ a penalty parameter. Define the stacked vector

$$\mathbf{v} = \begin{bmatrix} \mathbf{x} \\ \mathbf{y} \\ \mathbf{s} \end{bmatrix} \in \mathbb{R}^{M+P+U}.$$

The penalty term can be written as

$$\|\mathbf{A}\mathbf{x} + \mathbf{B}\mathbf{y} + \mathbf{s} - \mathbf{e}\|_2^2 = \|(\mathbf{A} \ \mathbf{B} \ \mathbf{I}_U)\mathbf{v} - \mathbf{e}\|_2^2 = \mathbf{v}^\top \mathbf{M}^\top \mathbf{M} \mathbf{v} - 2\mathbf{e}^\top \mathbf{M} \mathbf{v} + \mathbf{e}^\top \mathbf{e},$$

where $\mathbf{M} = [\mathbf{A} \ \mathbf{B} \ \mathbf{I}_U] \in \mathbb{R}^{U \times (M+P+U)}$.

Hence, the total objective (43) can be written as a quadratic function in $\mathbf{v}$:

$$F(\mathbf{v}) = \mathbf{v}^\top \mathbf{H}_v \mathbf{v} + \mathbf{h}_v^\top \mathbf{v} + \text{const}, \quad (44)$$

with

$$\mathbf{H}_v = \mathbf{H}_{\text{obj}} + \lambda \mathbf{M}^\top \mathbf{M}, \quad (45)$$

and

$$\mathbf{h}_v = \mathbf{h}_{\text{obj}} - 2\lambda \mathbf{M}^\top \mathbf{e}. \quad (46)$$

The objective-only contribution is

$$\mathbf{H}_{\text{obj}} = \begin{bmatrix} \mathbf{Q}_1 & \frac{1}{2}\mathbf{Q}_3 & 0 \\ \frac{1}{2}\mathbf{Q}_3^\top & \mathbf{Q}_2 & 0 \\ 0 & 0 & 0 \end{bmatrix}, \quad \mathbf{h}_{\text{obj}} = \begin{bmatrix} \mathbf{c} \\ \mathbf{d} \\ 0 \end{bmatrix},$$

since

$$\begin{bmatrix} \mathbf{x} \\ \mathbf{y} \\ \mathbf{s} \end{bmatrix}^\top \mathbf{H}_{\text{obj}} \begin{bmatrix} \mathbf{x} \\ \mathbf{y} \\ \mathbf{s} \end{bmatrix} = \mathbf{x}^\top \mathbf{Q}_1 \mathbf{x} + \mathbf{y}^\top \mathbf{Q}_2 \mathbf{y} + \mathbf{x}^\top \mathbf{Q}_3 \mathbf{y}.$$

The penalty contribution is

$$\mathbf{M}^\top \mathbf{M} = \begin{bmatrix} \mathbf{A}^\top \mathbf{A} & \mathbf{A}^\top \mathbf{B} & \mathbf{A}^\top \\ \mathbf{B}^\top \mathbf{A} & \mathbf{B}^\top \mathbf{B} & \mathbf{B}^\top \\ \mathbf{A} & \mathbf{B} & \mathbf{I}_U \end{bmatrix}, \quad \mathbf{M}^\top \mathbf{e} = \begin{bmatrix} \mathbf{A}^\top \mathbf{e} \\ \mathbf{B}^\top \mathbf{e} \\ \mathbf{e} \end{bmatrix}.$$

Therefore, the full quadratic and linear coefficients in (44) are given by (45)–(46).

Encoding matrices and affine map to binary variables. We now express $(\mathbf{x}, \mathbf{y}, \mathbf{s})$ in terms of binary variables. For the continuous and slack variables, we use the encoding schemes of Section 2. Specifically:

- Each continuous variable x_m is scaled to $\tilde{x}_m \in [0, 1]$ and encoded via a digit-wise scheme with precision J_m , which yields an affine mapping

$$\mathbf{x}^c = \mathbf{x}^L + \mathbf{B}^x \mathbf{z}^x,$$

where $\mathbf{x}^c \in \mathbb{R}^M$ collects the continuous components of $\mathbf{x}$, $\mathbf{x}^L \in \mathbb{R}^M$ is the vector of lower bounds, $\mathbf{B}^x \in \mathbb{R}^{M \times N_x}$ is the encoding matrix, and $\mathbf{z}^x \in \{0, 1\}^{N_x}$ is the corresponding binary vector.

- The binary/integer variables $\mathbf{y}$ are already in $\{0, 1\}^P$ and enter the QUBO directly.
- Each slack variable s_u is scaled to $\tilde{s}_u \in [0, 1]$ with $s_u = s_u^{\max} \tilde{s}_u$ and encoded similarly, yielding

$$\mathbf{s} = \mathbf{B}^s \mathbf{z}^s,$$

where $\mathbf{B}^s \in \mathbb{R}^{U \times N_s}$ and $\mathbf{z}^s \in \{0, 1\}^{N_s}$.

Collect all binary variables into a single vector

$$\mathbf{z} = \begin{bmatrix} \mathbf{z}^x \\ \mathbf{y} \\ \mathbf{z}^s \end{bmatrix} \in \{0, 1\}^\beta, \quad \beta = N_x + P + N_s.$$

We define the affine map from $\mathbf{z}$ to $\mathbf{v}$ as

$$\mathbf{v} = \boldsymbol{\alpha} + \mathbf{B}_z \mathbf{z}, \tag{47}$$

where

$$\boldsymbol{\alpha} = \begin{bmatrix} \mathbf{x}^L \\ \mathbf{0} \\ \mathbf{0} \end{bmatrix}, \quad \mathbf{B}_z = \begin{bmatrix} \mathbf{B}^x & 0 & 0 \\ 0 & \mathbf{I}_P & 0 \\ 0 & 0 & \mathbf{B}^s \end{bmatrix}.$$

Quadratic form in the binary variables. Substituting (47) into (44), we obtain

$$F(\mathbf{z}) = F(\boldsymbol{\alpha} + \mathbf{B}_z \mathbf{z})$$

$$\begin{aligned}
&= (\boldsymbol{\alpha} + \mathbf{B}_z \mathbf{z})^\top \mathbf{H}_v (\boldsymbol{\alpha} + \mathbf{B}_z \mathbf{z}) + \mathbf{h}_v^\top (\boldsymbol{\alpha} + \mathbf{B}_z \mathbf{z}) + \text{const} \\
&= \mathbf{z}^\top \mathbf{B}_z^\top \mathbf{H}_v \mathbf{B}_z \mathbf{z} + (2\boldsymbol{\alpha}^\top \mathbf{H}_v \mathbf{B}_z + \mathbf{h}_v^\top \mathbf{B}_z) \mathbf{z} + \text{const}.
\end{aligned}$$

Define the quadratic and linear coefficients in terms of $\mathbf{z}$ as

$$\mathbf{Q}^{(2)} := \mathbf{B}_z^\top \mathbf{H}_v \mathbf{B}_z, \quad \mathbf{q}^{(1)} := \mathbf{B}_z^\top (\mathbf{h}_v + 2\mathbf{H}_v \boldsymbol{\alpha}). \quad (48)$$

Then

$$F(\mathbf{z}) = \mathbf{z}^\top \mathbf{Q}^{(2)} \mathbf{z} + (\mathbf{q}^{(1)})^\top \mathbf{z} + \text{const}. \quad (49)$$

To obtain a pure QUBO of the form $\mathbf{z}^\top \mathbf{Q} \mathbf{z}$ with no explicit linear term, we exploit the fact that $z_i^2 = z_i$ for all binary variables. Writing

$$(\mathbf{q}^{(1)})^\top \mathbf{z} = \sum_{i=1}^{\beta} q_i^{(1)} z_i = \sum_{i=1}^{\beta} q_i^{(1)} z_i^2,$$

we can absorb the linear term into the diagonal of the quadratic form by defining

$$\mathbf{Q} = \mathbf{Q}^{(2)} + \frac{1}{2} \text{diag}(\mathbf{q}^{(1)}). \quad (50)$$

Then $\mathbf{z}^\top \mathbf{Q} \mathbf{z}$ and $\mathbf{z}^\top \mathbf{Q}^{(2)} \mathbf{z} + (\mathbf{q}^{(1)})^\top \mathbf{z}$ differ only by a constant, and thus share the same minimizers. Therefore, the final QUBO corresponding to the penalized MIQP (Eq. (43)) is

$$\min_{\mathbf{z} \in \{0,1\}^\beta} \mathbf{z}^\top \mathbf{Q} \mathbf{z}, \quad (51)$$

with $\mathbf{Q}$ given by Eq. (50) and Eq. (48).

Block structure of the QUBO matrix. By construction, $\mathbf{z}$ is partitioned as $\mathbf{z} = (\mathbf{z}^x, \mathbf{y}, \mathbf{z}^s)$, and $\mathbf{Q}$ inherits a block structure:

$$\mathbf{Q} = \begin{bmatrix} \mathbf{Q}^{xx} & \mathbf{Q}^{xy} & \mathbf{Q}^{xs} \\ (\mathbf{Q}^{xy})^\top & \mathbf{Q}^{yy} & \mathbf{Q}^{ys} \\ (\mathbf{Q}^{xs})^\top & (\mathbf{Q}^{ys})^\top & \mathbf{Q}^{ss} \end{bmatrix},$$

where:

- $Q^{xx} \in \mathbb{R}^{N_x \times N_x}$ contains all interactions among the binary digits of the continuous variables;
- $Q^{yy} \in \mathbb{R}^{P \times P}$ collects the quadratic terms in the original binary variables y (including contributions from the penalty);
- $Q^{ss} \in \mathbb{R}^{N_s \times N_s}$ contains all interactions among the binary digits of the slack variables;
- Q^{xy} , Q^{xs} , and Q^{ys} capture the cross-interactions between continuous, binary, and slack blocks.

A.3 Penalty-weight selection

Consider an encoded problem of the form

$$\min_{\mathbf{z} \in \{0,1\}^{N_z}} F(\mathbf{z}) + \sum_{k \in \mathcal{K}} \lambda_k r_k(\mathbf{z})^2,$$

where $F(\mathbf{z})$ is the encoded objective *before* adding constraint penalties, and $r_k(\mathbf{z})$ is the residual of constraint k after variable encoding and slack-variable introduction. The goal is to choose λ_k large enough to discourage constraint violation, while avoiding unnecessarily large QUBO coefficients. We first bound the range of the unpenalized objective. Let

$$F(\mathbf{z}) = c + \sum_i q_i z_i + \sum_{i < j} q_{ij} z_i z_j.$$

A conservative lower and upper bound over the binary domain are

$$F_L^U = c + \sum_i \min\{q_i, 0\} + \sum_{i < j} \min\{q_{ij}, 0\},$$

$$F_U^U = c + \sum_i \max\{q_i, 0\} + \sum_{i < j} \max\{q_{ij}, 0\}.$$

Thus, an *upper* bound on the objective range is

$$\Delta_F^U = F_U^U - F_L^U.$$

This bound is loose because it treats all terms independently, but it is easy to compute from the QUBO coefficients. Next, we bound the scale of each constraint residual. Formally, for

each encoded residual, we write

$$r_k(\mathbf{z}) = \alpha_k + \sum_i \beta_{ki} z_i + \sum_{i < j} \gamma_{kij} z_i z_j.$$

A conservative bound on its maximum absolute value is

$$R_k^U = |\alpha_k| + \sum_i |\beta_{ki}| + \sum_{i < j} |\gamma_{kij}|.$$

Then, for a user-specified feasibility tolerance, $\eta_k > 0$, we can compute a conservative penalty. If a point violates at least one constraint by $(|r_k(\mathbf{z})| \geq \eta_k)$, then its normalized squared penalty is at least

$$\delta = \min_{k \in \mathcal{K}} \left(\frac{\eta_k}{R_k^U} \right)^2.$$

Therefore, a conservative penalty choice is

$$\lambda_{\text{cons}} > \frac{\Delta_F^U}{\delta}.$$

The rationale is that the penalty assigned to any infeasible point should exceed the largest possible improvement in the original objective that could be obtained by violating constraints. This is a finite-domain exact-penalty argument and assumes that the penalized QUBO is solved to global optimality. Note that the value λ_{cons} can be very large. Such large penalties may increase the dynamic range of the QUBO coefficients and make the instance more sensitive to coefficient scaling, finite precision, and hardware noise. Therefore, one should treat it as an upper bound rather than as the default penalty. In practice, we select the smallest penalty that achieves the desired feasibility tolerance.

A.4 Penalty sensitivity for the Alan instance.

We performed a penalty-sensitivity study on the **Alan** instance. In this experiment, all penalty weights were set to a common value λ , and λ was varied over the set $\lambda \in \{50, 100, 250, 500, 1000, 2500, 5000\}$. For each value, we solved the constant-size zoom-in QUBO formulation with fixed encoding depth and recorded the original objective, maximum feasibility residual, individual equality residuals, inequality violations, and

QUBO coefficient scale. The maximum feasibility residual is defined as $\rho = \max_{1 \leq i \leq 7} \{|c_i|\}$. For each penalty value, we also record two measures of the unscaled QUBO coefficient magnitude. $Q_{\max}$ measures the largest unscaled coefficient magnitude, whereas $\text{dyn}(Q)$ measures the spread between the largest and smallest nonzero QUBO coefficients. These quantities are typical indicators of coefficient scaling and potential sensitivity to finite-precision implementation.

Table 9: Penalty-weight sensitivity for the **Alan** instance. Here ρ denotes the maximum feasibility residual, and $Q_{\max}$ and $\text{dyn}(Q)$ are computed before coefficient rescaling.

λ	Objective	ρ	$ e_1 $	$ e_2 $	$Q_{\max}$	$\text{dyn}(Q)$
50	1.7200	1.00×10^{-1}	1.00×10^{-1}	1.78×10^{-15}	2.97×10^3	5.94×10^3
100	2.1472	6.00×10^{-2}	6.00×10^{-2}	2.00×10^{-2}	8.00×10^2	2.00×10^4
250	3.1014	0.00	0.00	0.00	2.00×10^3	2.00×10^4
500	3.0000	0.00	0.00	0.00	2.97×10^4	5.95×10^3
1000	2.9462	4.05×10^{-3}	4.05×10^{-3}	7.02×10^{-4}	8.00×10^3	1.25×10^{10}
2500	3.2322	2.98×10^{-3}	2.98×10^{-3}	4.61×10^{-4}	2.00×10^4	3.91×10^9
5000	3.2480	0.00	0.00	0.00	4.00×10^4	2.00×10^4

The results illustrate the tradeoff involved in penalty selection. For small penalties, such as $\lambda = 50$ and $\lambda = 100$, the solver obtains lower objective values but leaves non-negligible feasibility residuals. Increasing the penalty to $\lambda = 250$ – 500 enforces feasibility; among these strictly feasible solutions, $\lambda = 500$ gives the lower original objective. Further increasing the penalty does not monotonically improve the original objective and can increase the coefficient scale or dynamic range of the QUBO. Therefore, $\lambda = 500$ is a reasonable working penalty for this illustrative experiment, but the sensitivity also confirms that penalty selection is problem-dependent and should be treated as a tuning step.